



Universidad
del País Vasco

Euskal Herriko
Unibertsitatea

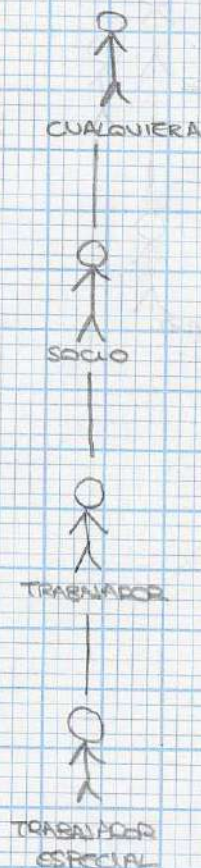
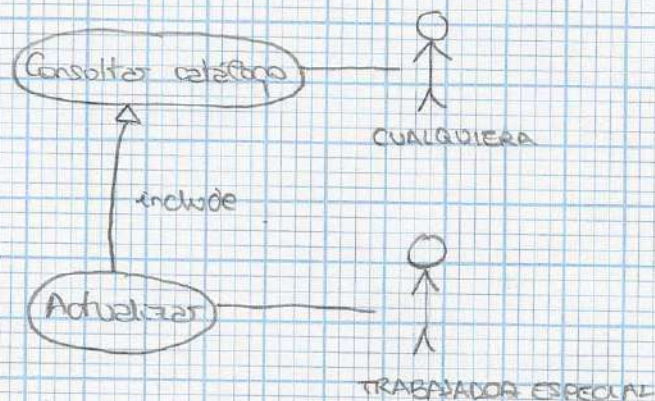
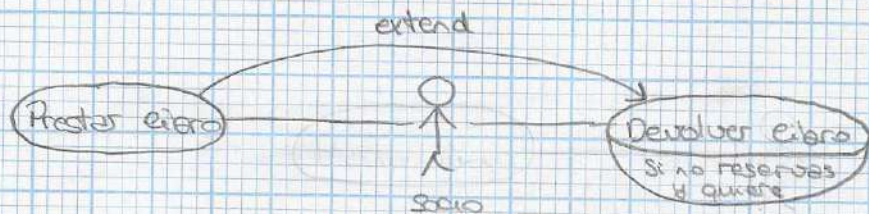
ADSI

IKER FERNÁNDEZ MOLANO

EJERCICIO 1

CASO DE USO ????

- Prestar libro 3 días } Socio (P.A.)
- Prestar libro 3 semanas }
- ✓ - Prestar revista TRABAJADOR (P.A.)
- x - Almacenar préstamo libro
- x - Almacenar préstamo revista } (son parte del préstamo x caso uso)
- ✓ - Devolver libro
- ✓ - Devolver revista } (prestar libro se hace en otro tiempo)
- x - Reservar libro (dentro del préstamo)
- x - Extender fecha préstamo libro (Relación)
- ✓ - Consultar catálogo CUALQUIERA (P.A.)
- ✓ - Actualizar catálogo TRABAJADOR ESPECIAL (P.A.)



EXERCICIO 2

Identificación

Realizar consultas

Consultar horario

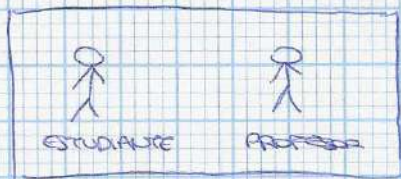
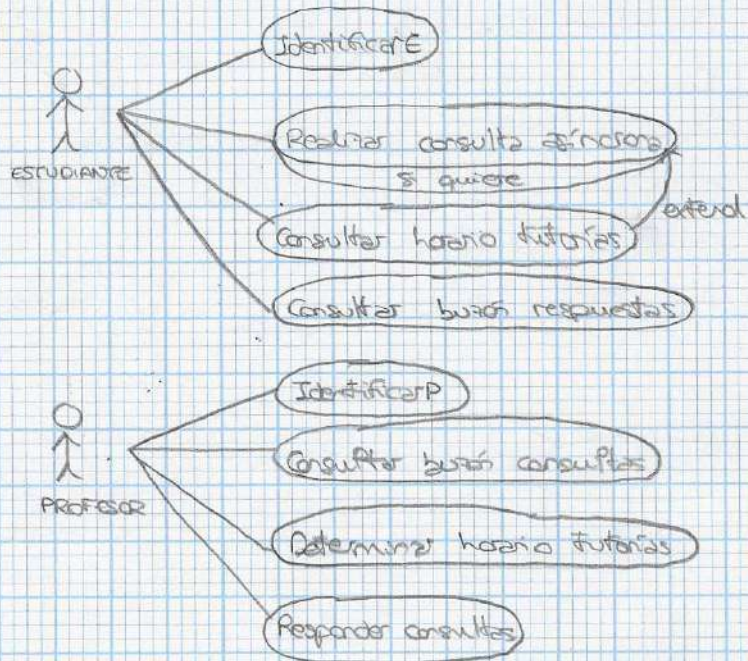
Determinar horario tutorías

Responder consultas

Usuario

Alumno

Profesor



EJERCICIO 4

Hacer pedido

Analizar viabilidad pedido

Preparar entrega

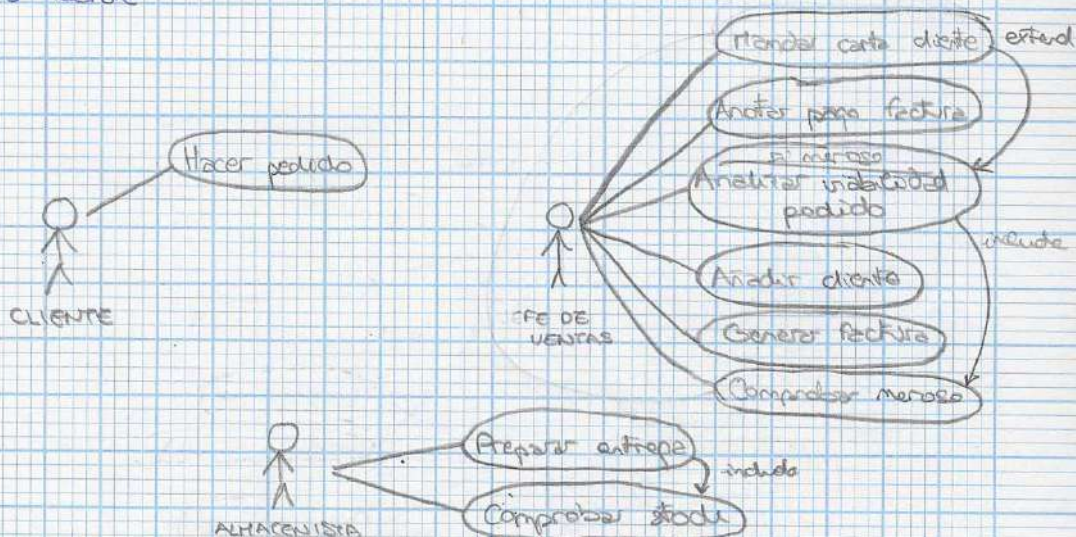
Generar factura

Vendedor

Cliente

Jefe de Ventas

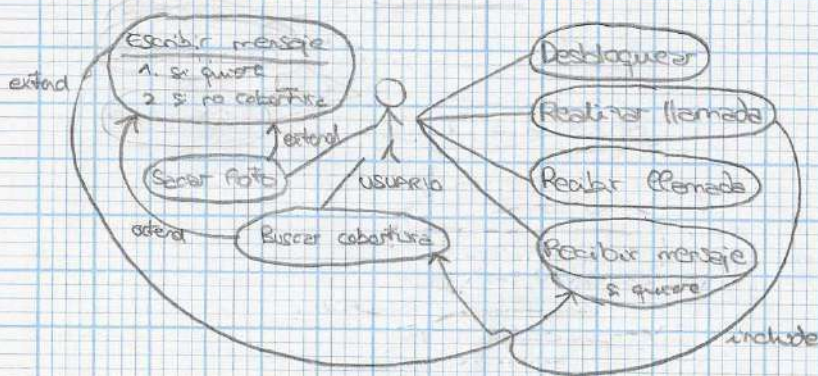
Almacenista



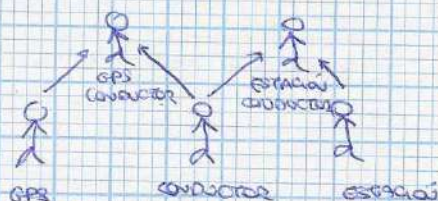
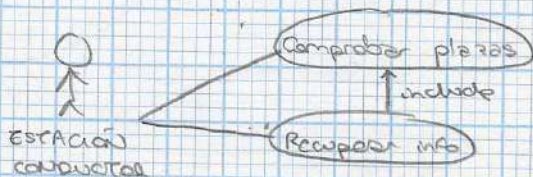
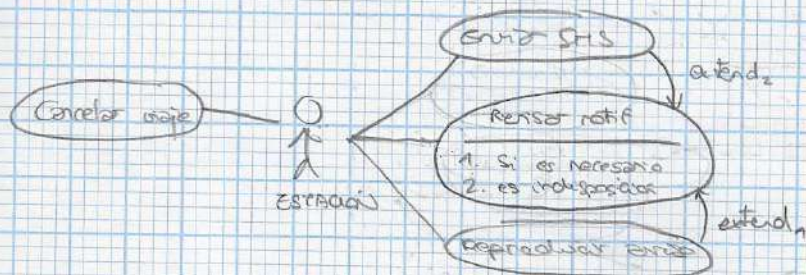
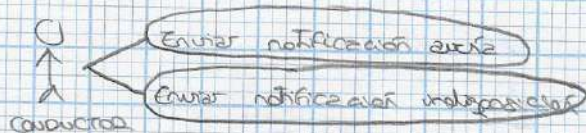
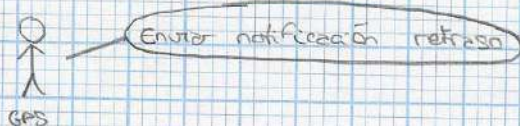
EJERCICIO 10

- ✓ Introducir contraseña
- ✓ Realizar llamada
- ✓ Recibir llamada
- ✓ Enviar mensaje
- ✗ Regular volumen
- ✓ Sacar foto

Usuario



EJERCICIO 7



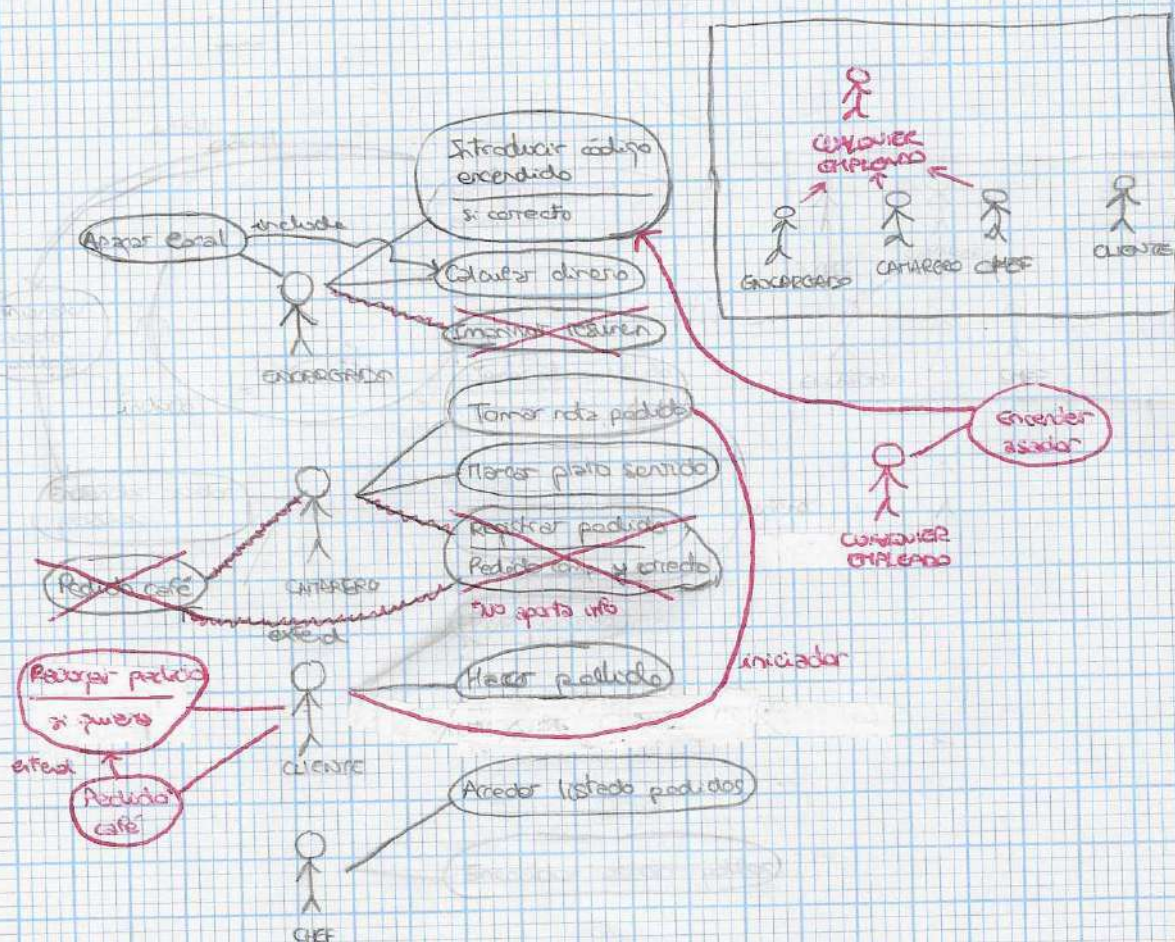
EJERCICIO 29

CASOS ?

- Introducir código encendido
- Tomar nota pedidos
- Marcar plato servido
- Hacer pedido
- Registrar pedido
- Acceder estado pedidos
- Calcular dinero
- Imprimir resumen
- Encender asador pellos
- Apagar cocca

ACTORES ?

- Encargado
- Camarero
- Cliente
- Chef



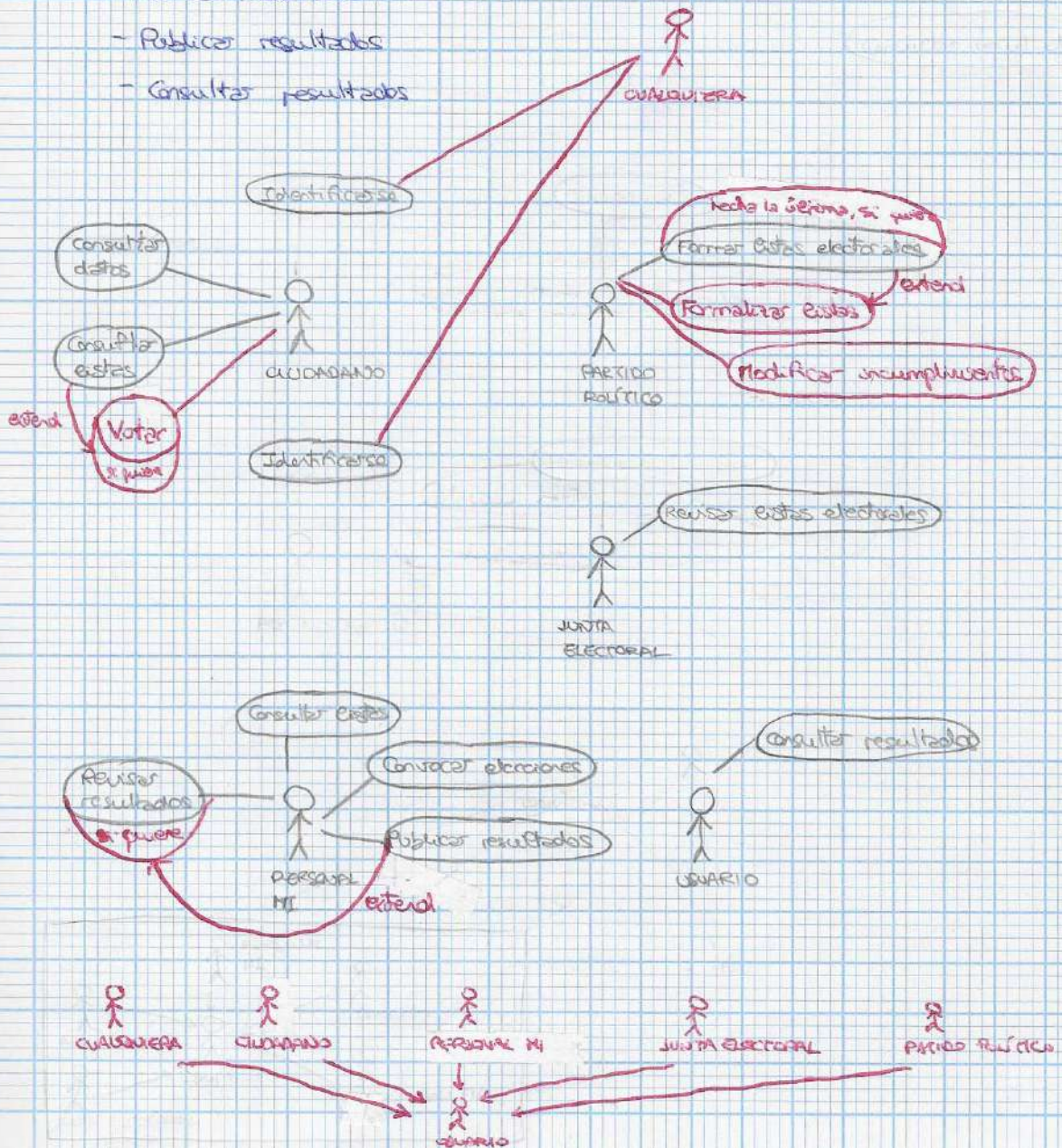
EXERCICIO 34

CASOS?

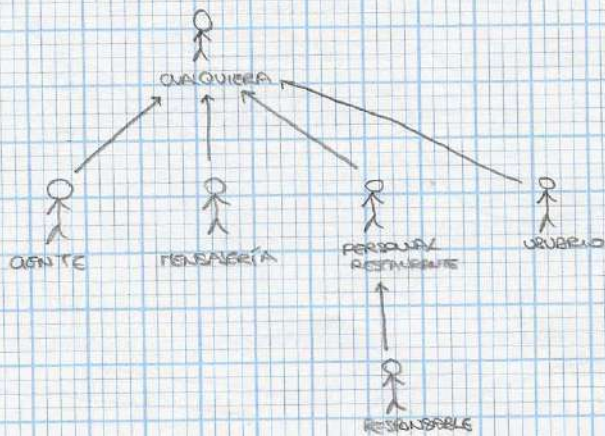
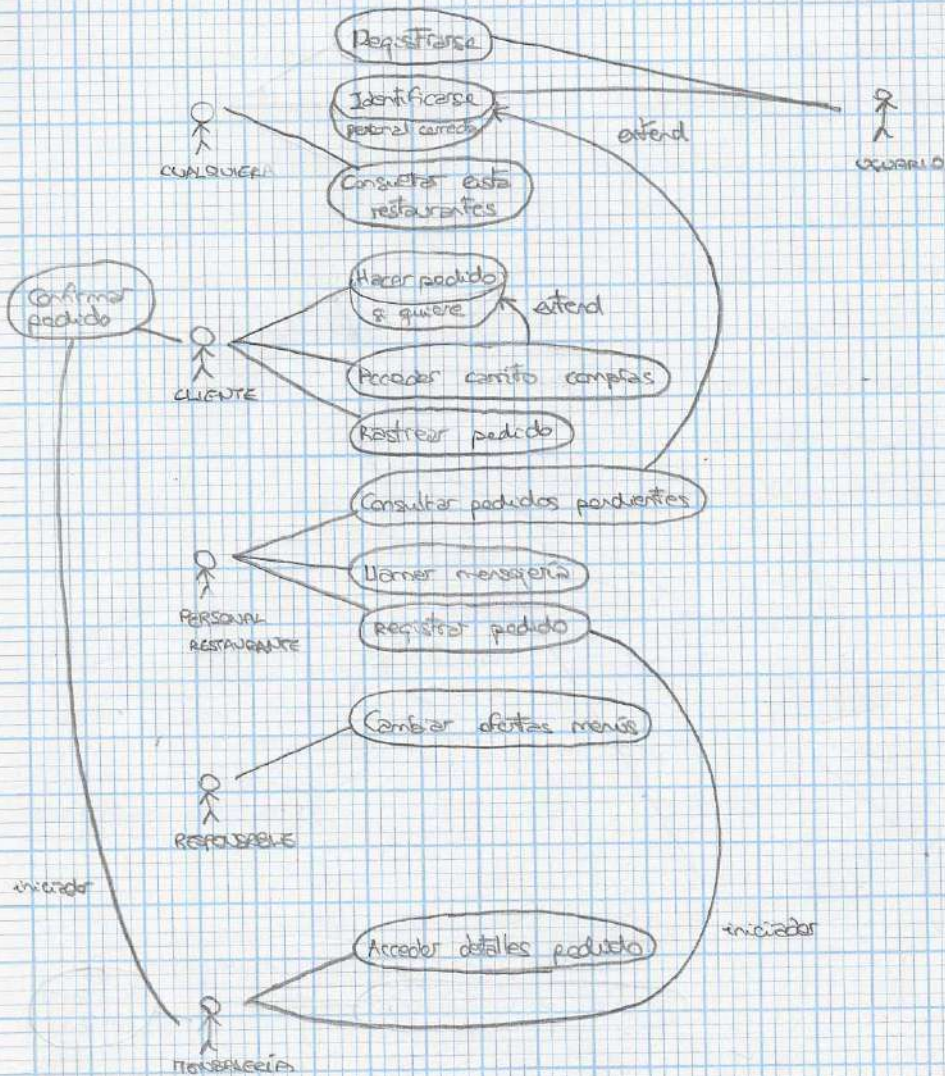
- Identificarse
- Formar listas electorales
- Revisar listas electorales
- Consultar listas
- Consultar datos personales
- Convocar elecciones
- Revisar resultados
- Publicar resultados
- Consultar resultados

ACTORES?

- ciudadano
- partido político
- administración
- junta electoral
- personal TI
- usuario



EXERCICIO 40

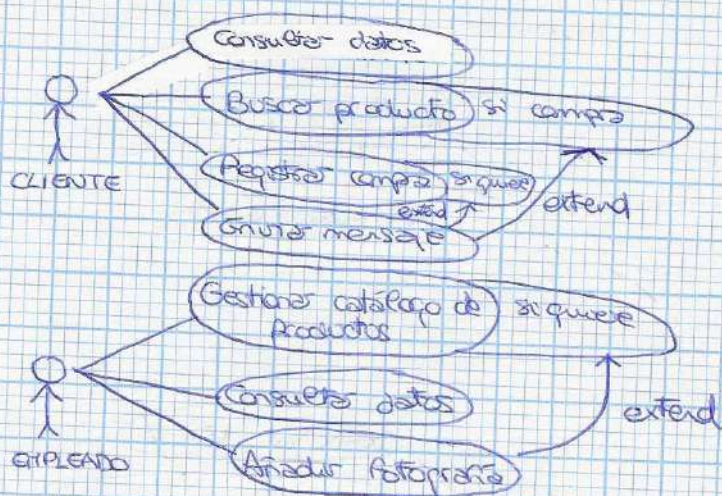
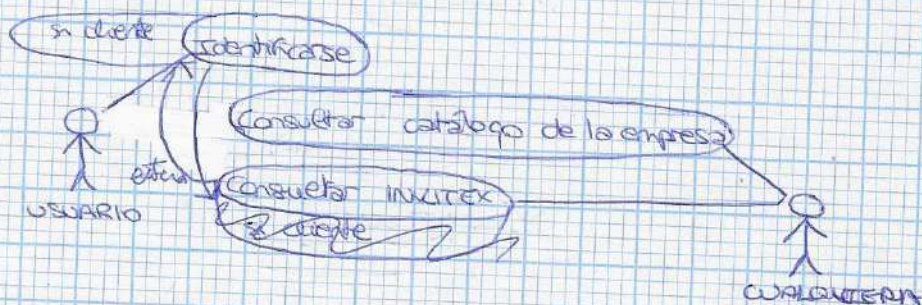


CASOS DE USO

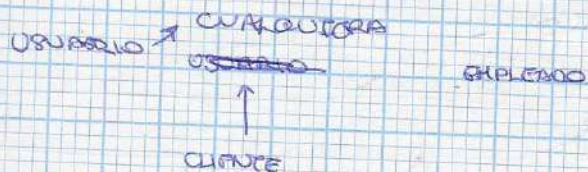
- Identificarse
- Gestionar catálogo de productos
- Consultar catálogo de la empresa
- Consultar Inuitex

ACTORES

- USUARIO
- CLIENTE
- EMPLEADO



JERARQUÍA

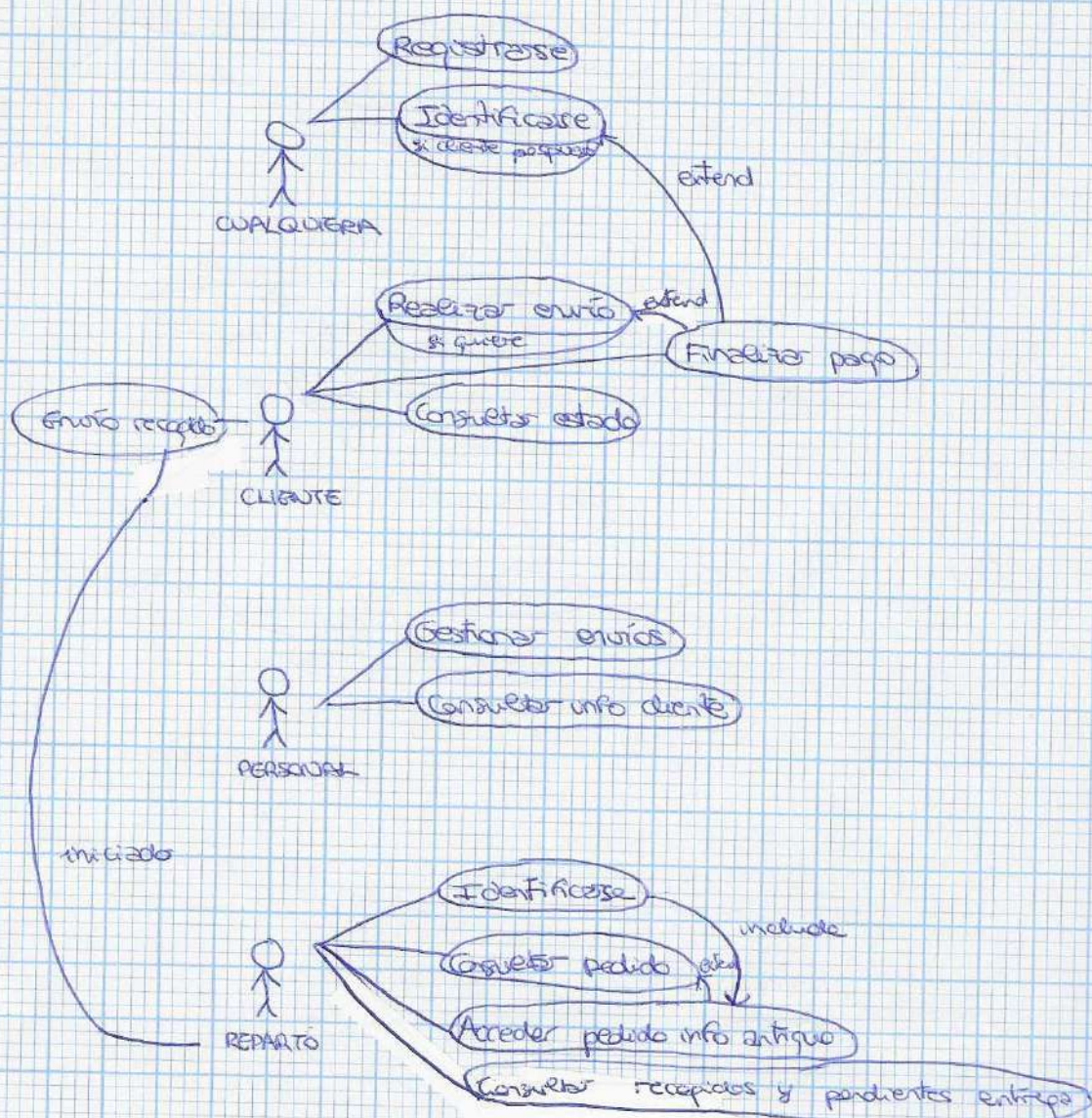


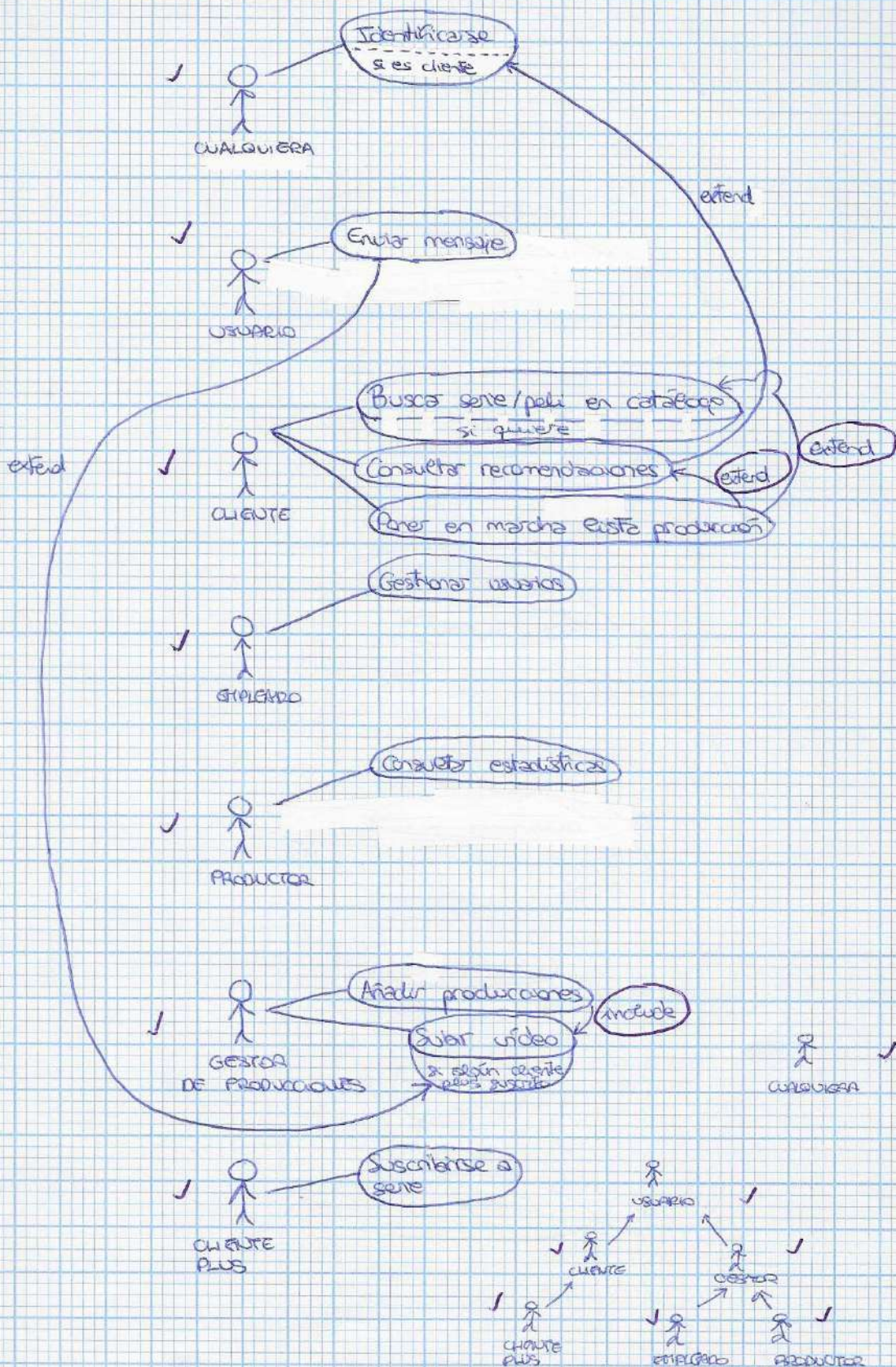
39

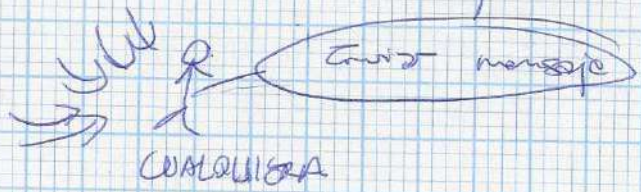
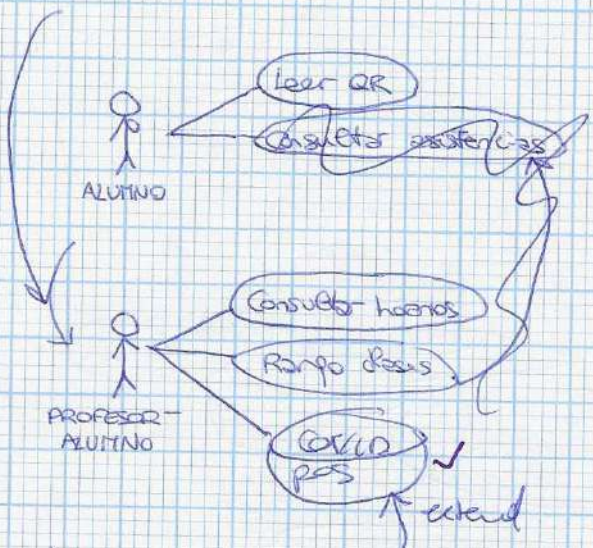
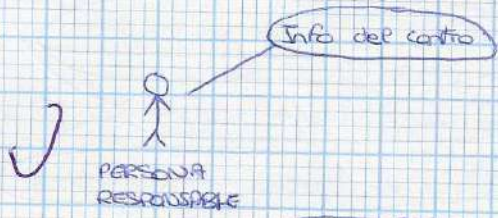
CASOS DE USO

ACTORES

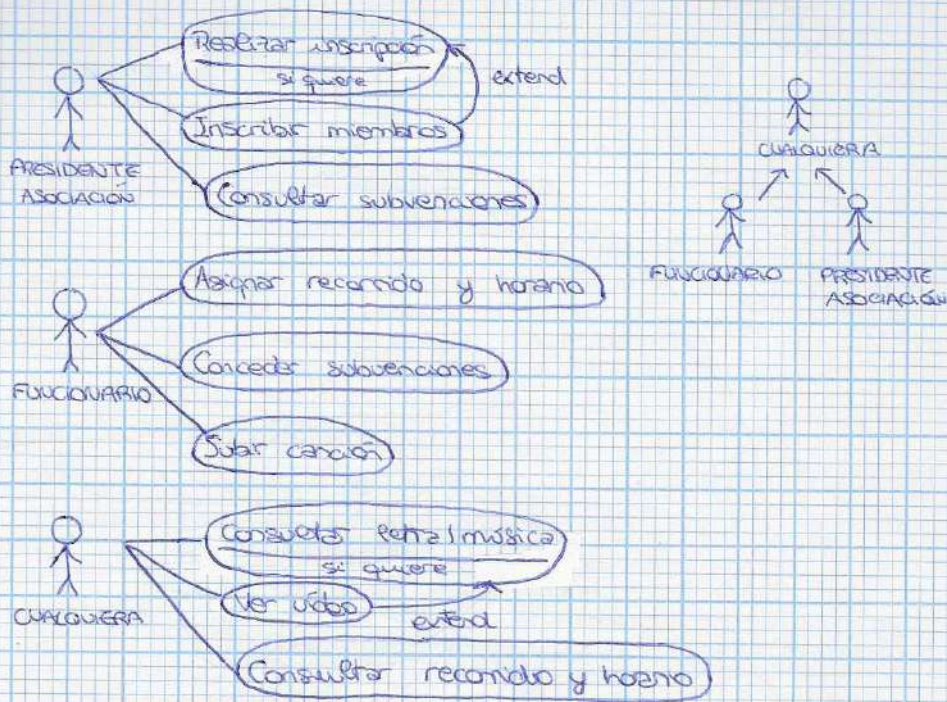
- Cualquiera
- Cliente
- Personal

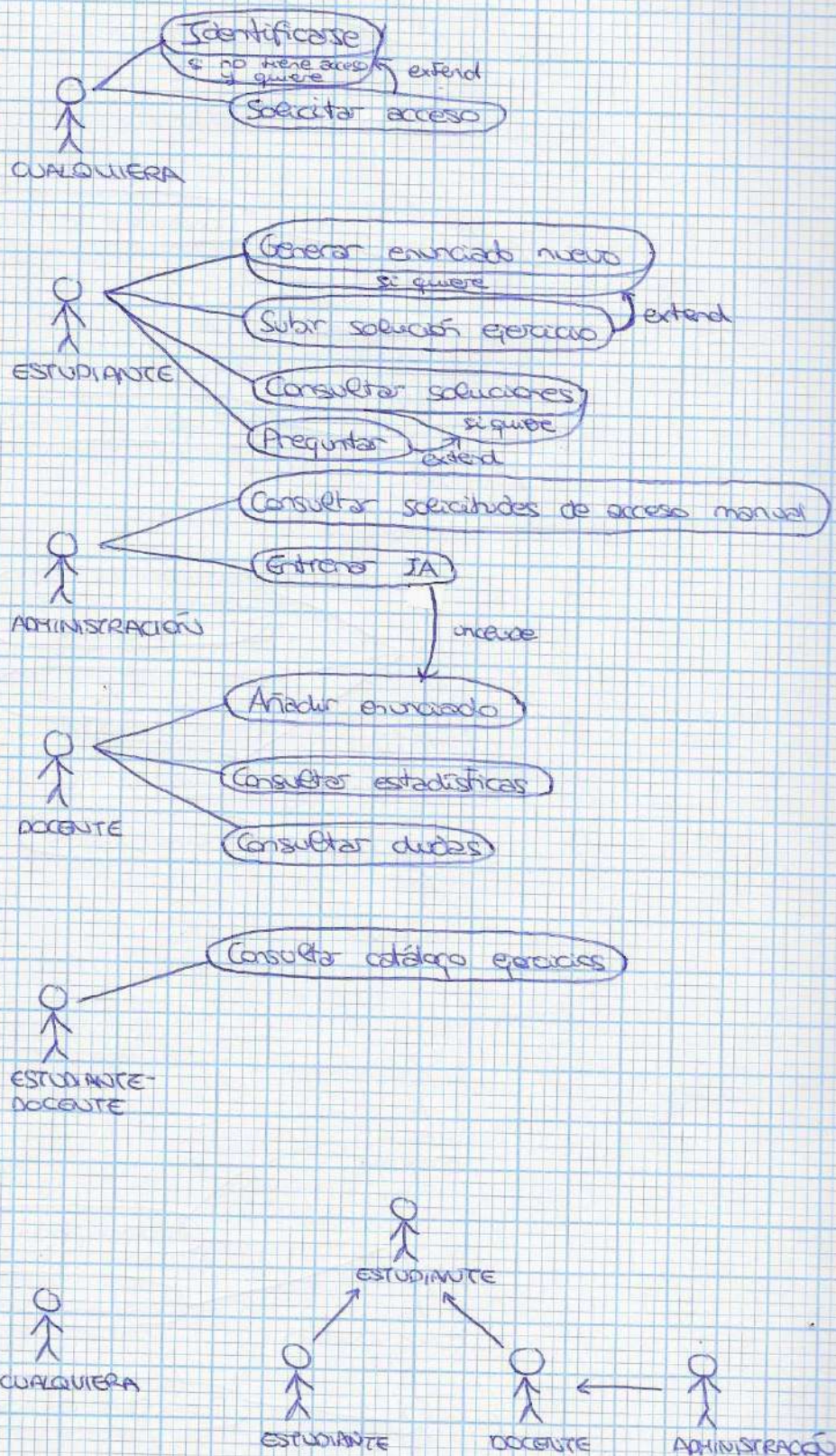


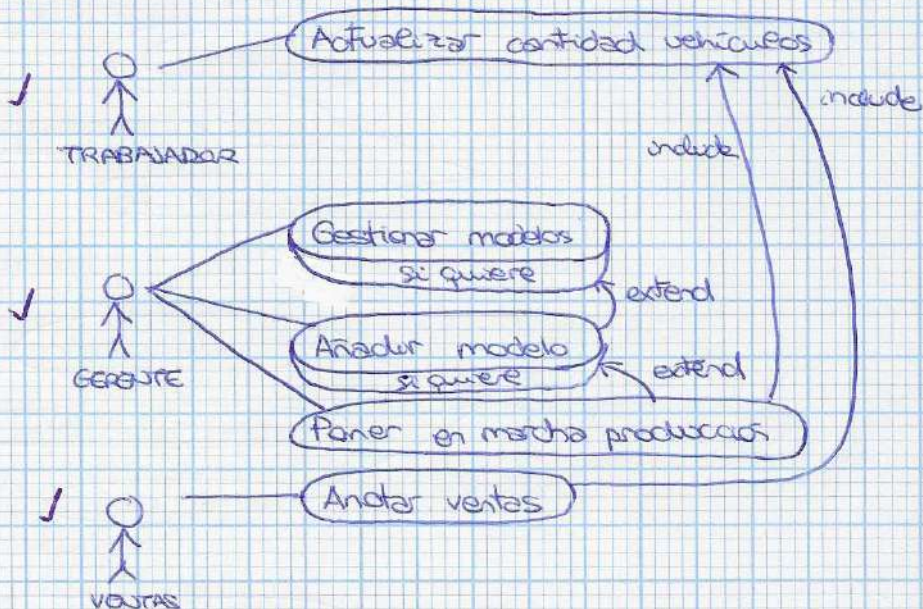
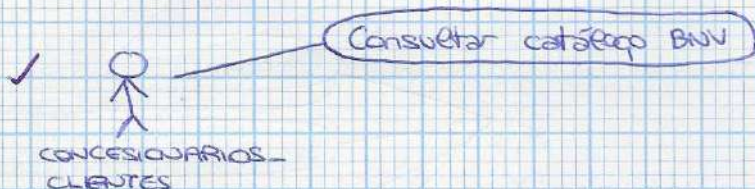


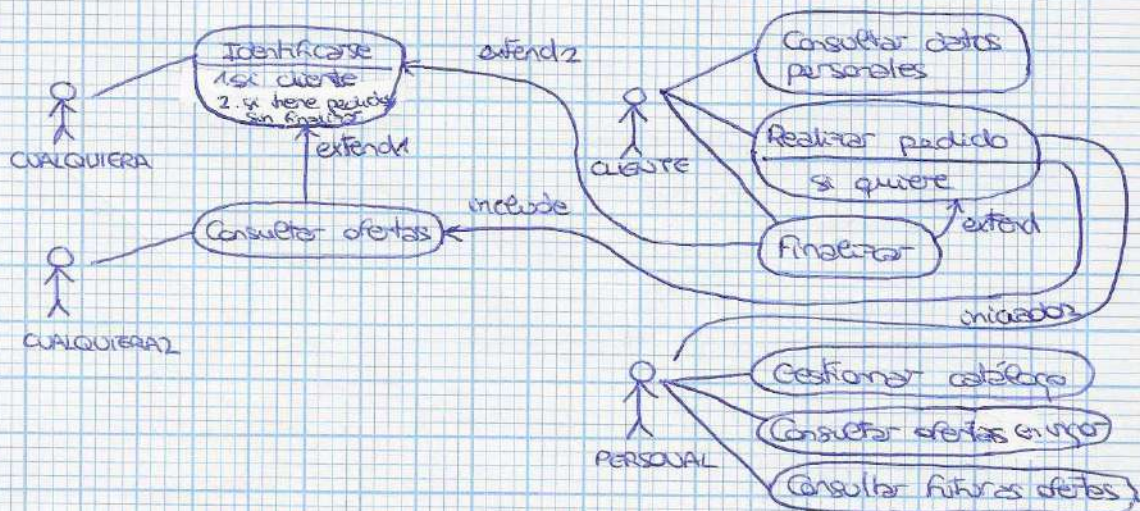
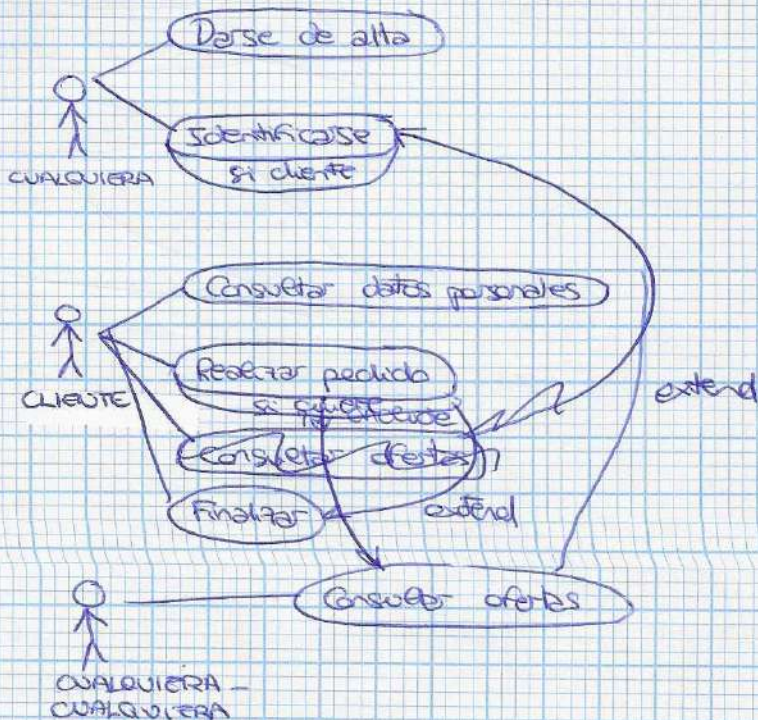


25

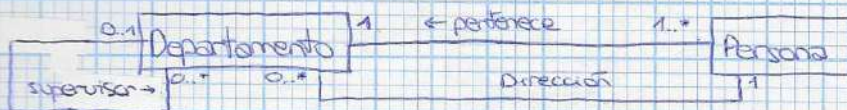




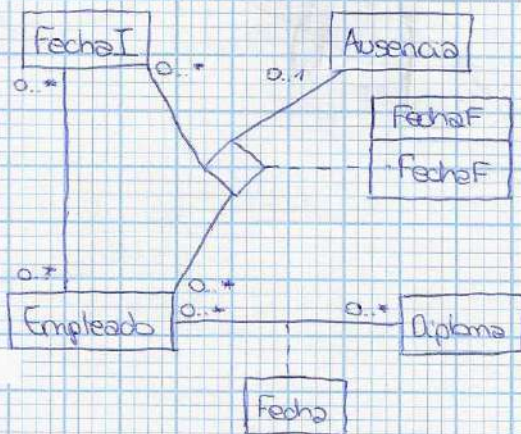
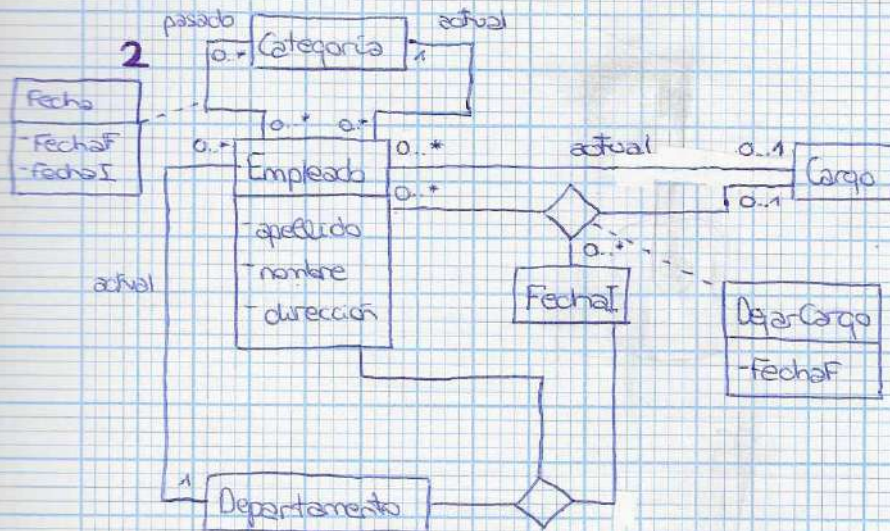




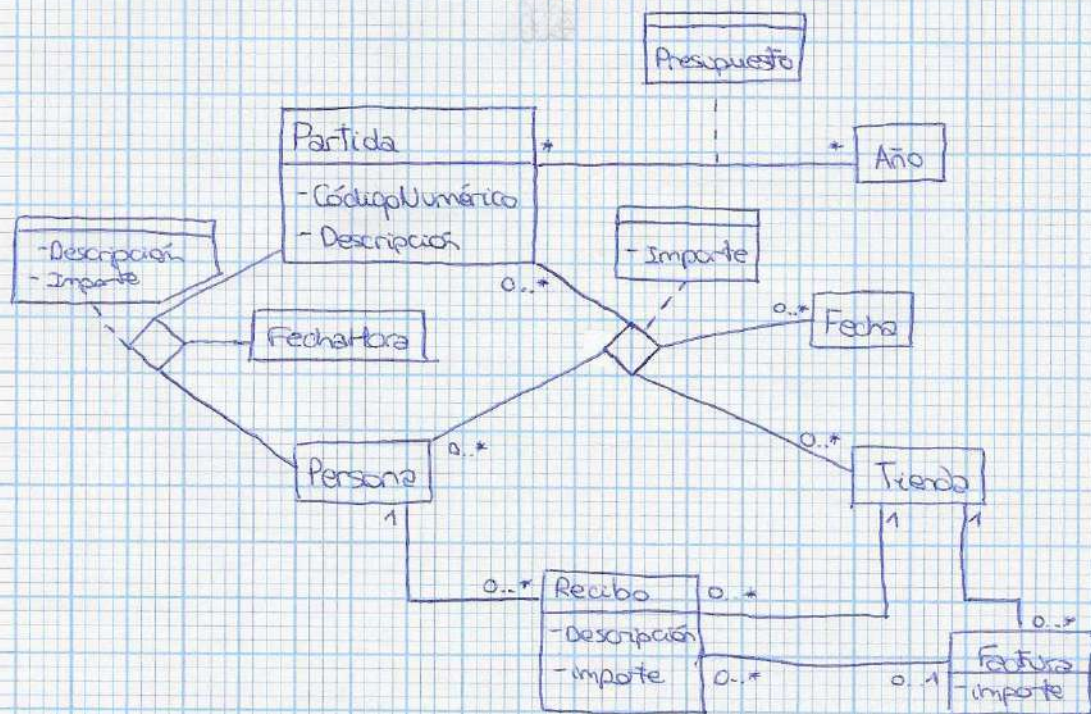
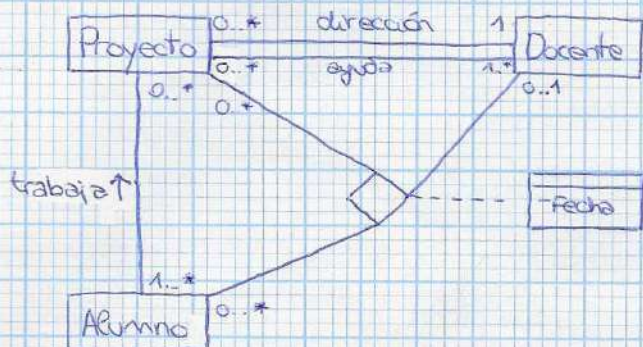
1

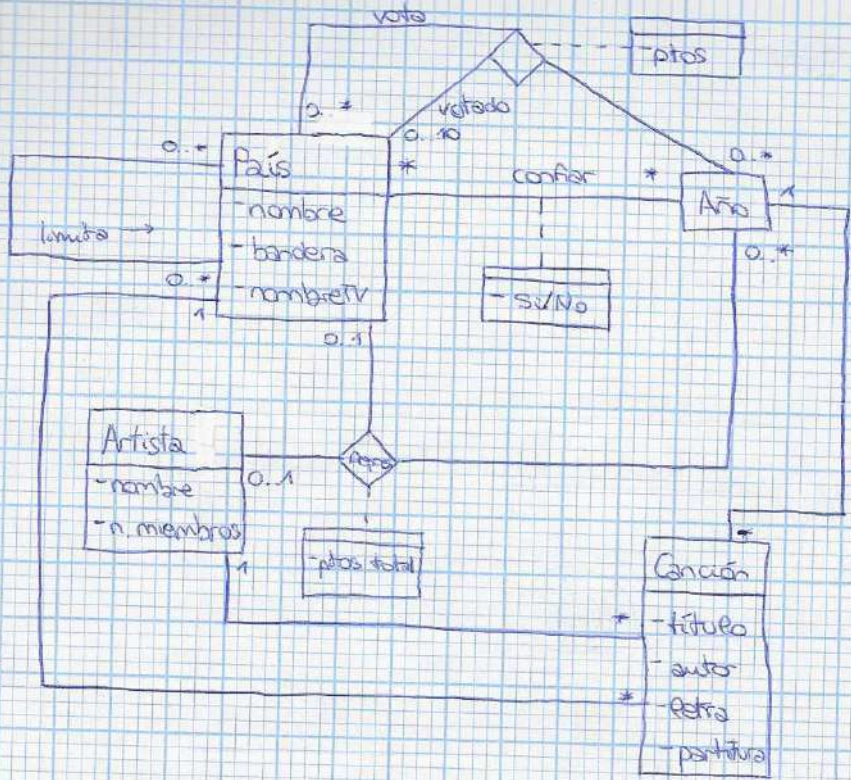


2

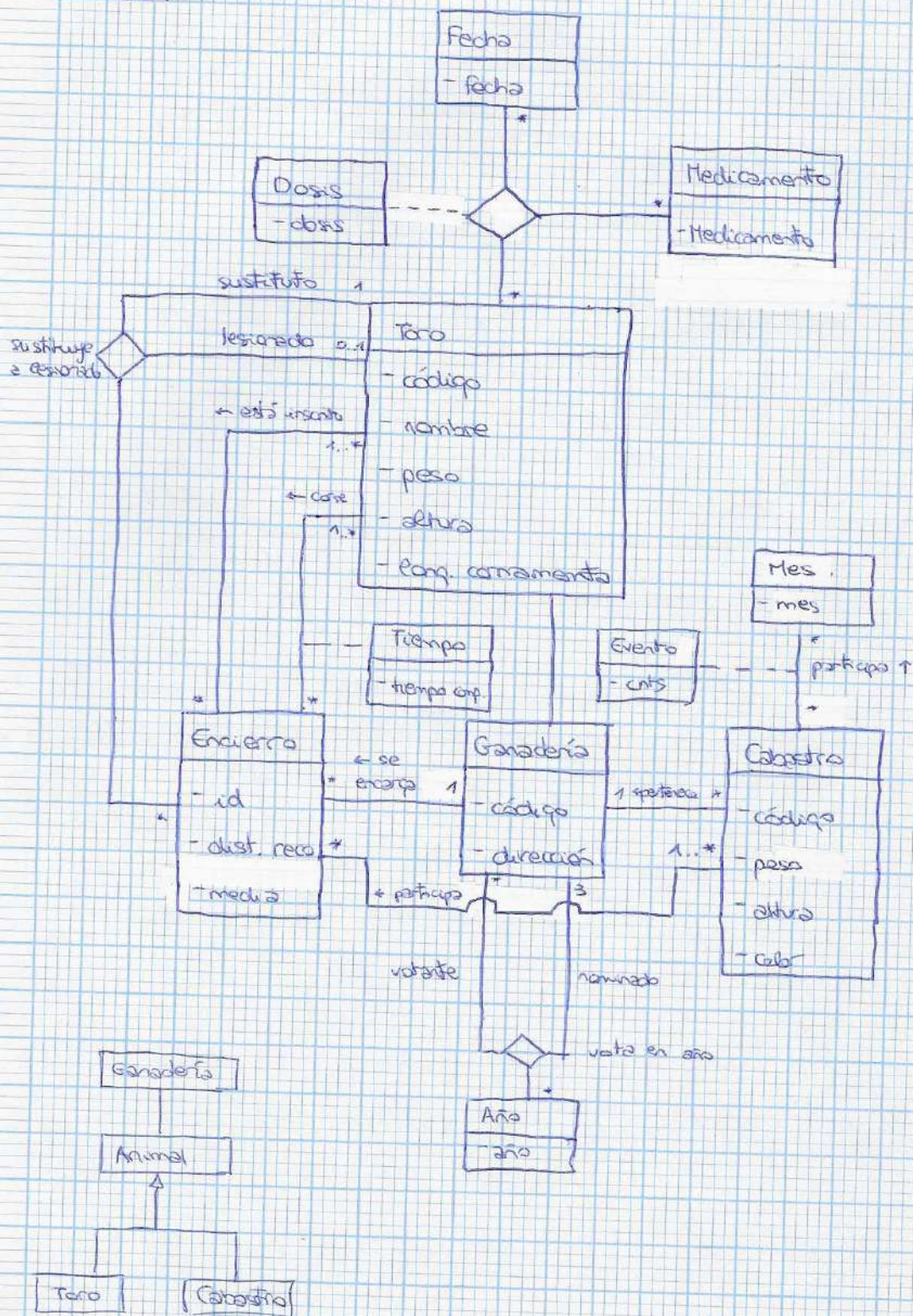


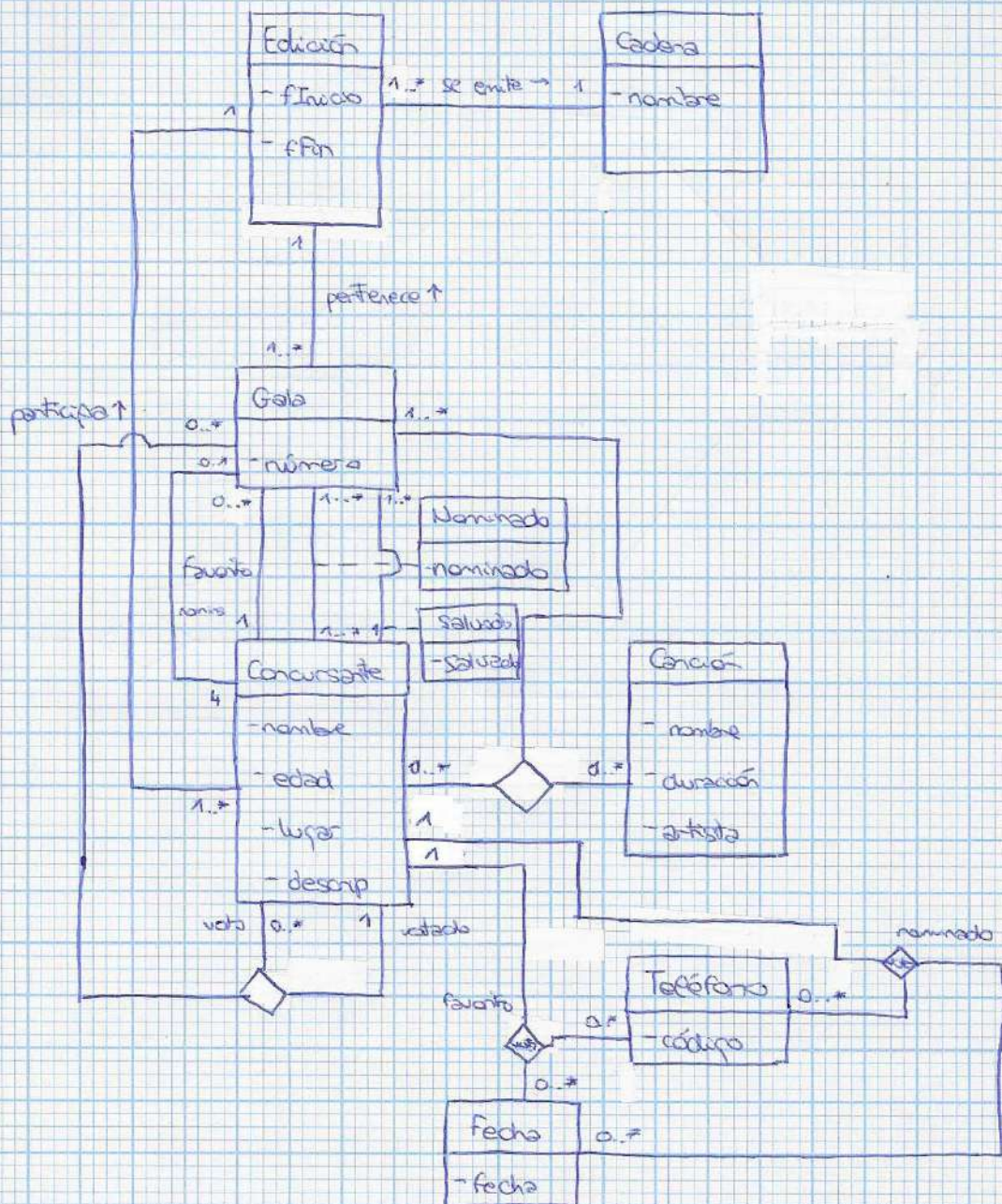
3



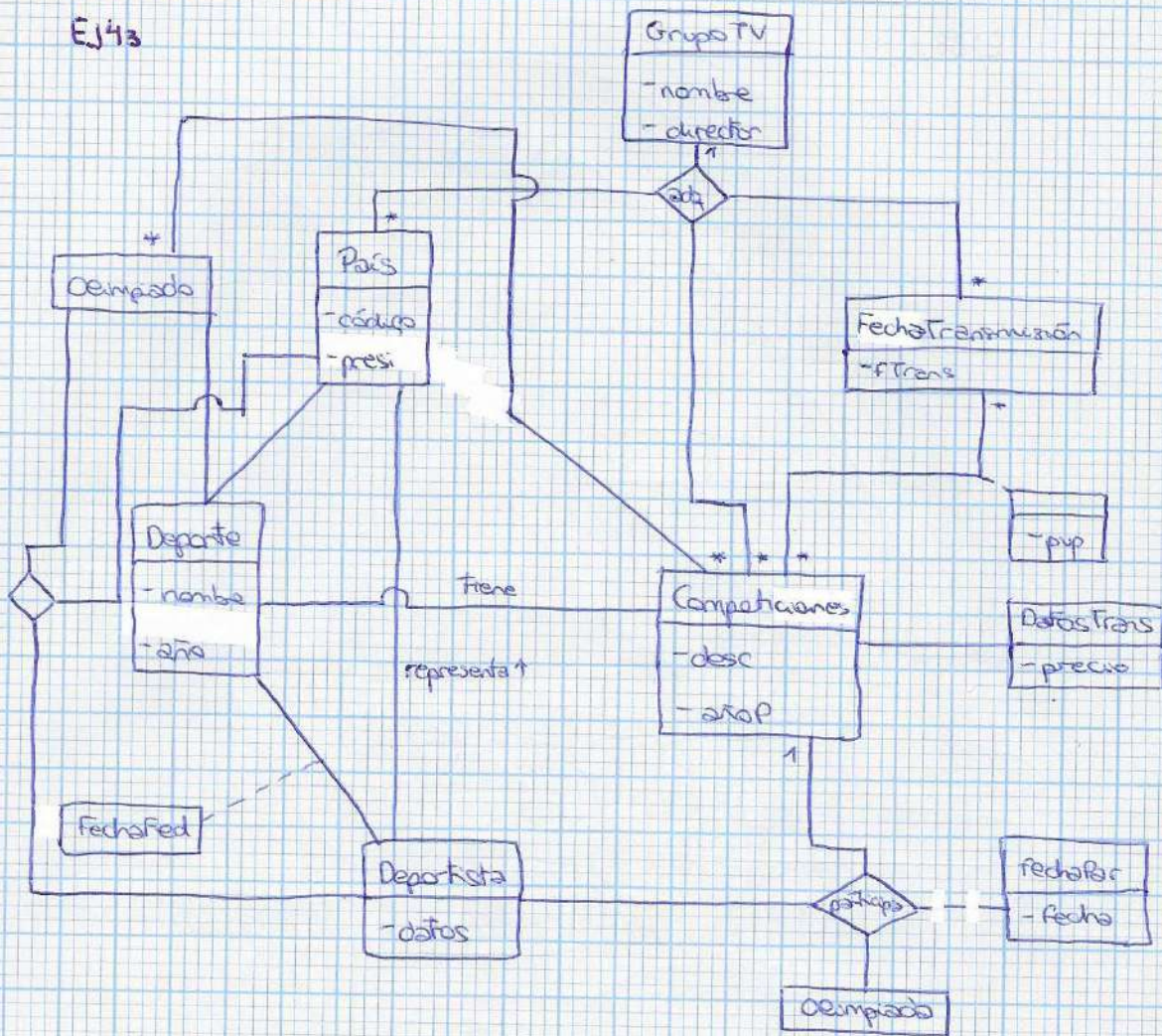


EJ 31



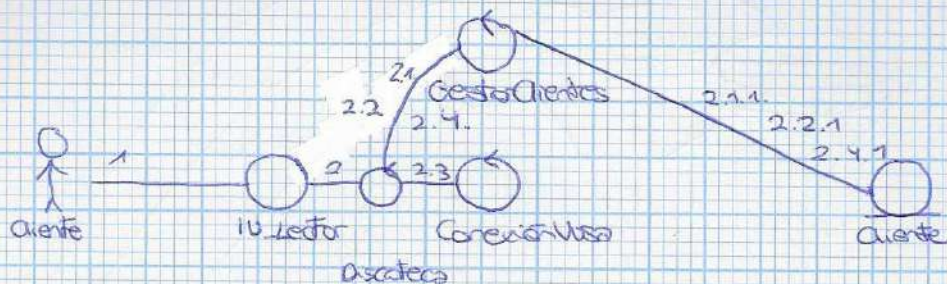


EJ43



EM.

ENTRADA DISCOTECA



1- obtener numTarjeta

2- EntrarDiscoteca (NumTarjeta) : void

2.1.- elCliente = buscarCliente(numTarjeta) : Cliente

2.1.1. comprobCod(numTarjeta) : bool

2.2.- obtenerDatosVisa (elCliente) : JSOU + num : int, cad : Date,

nom : String +

2.2.1.- obtenerDatosVisa() : JSOU + num : int, cad : Date,

nom : String +

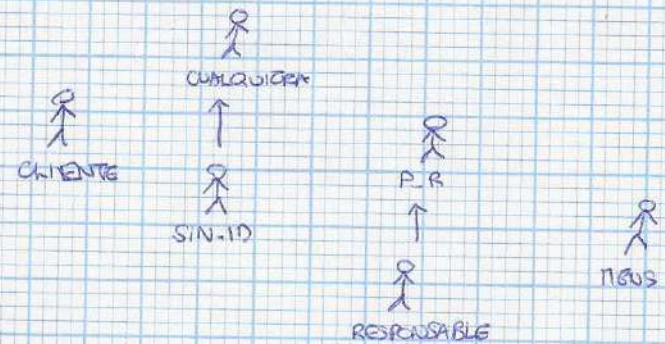
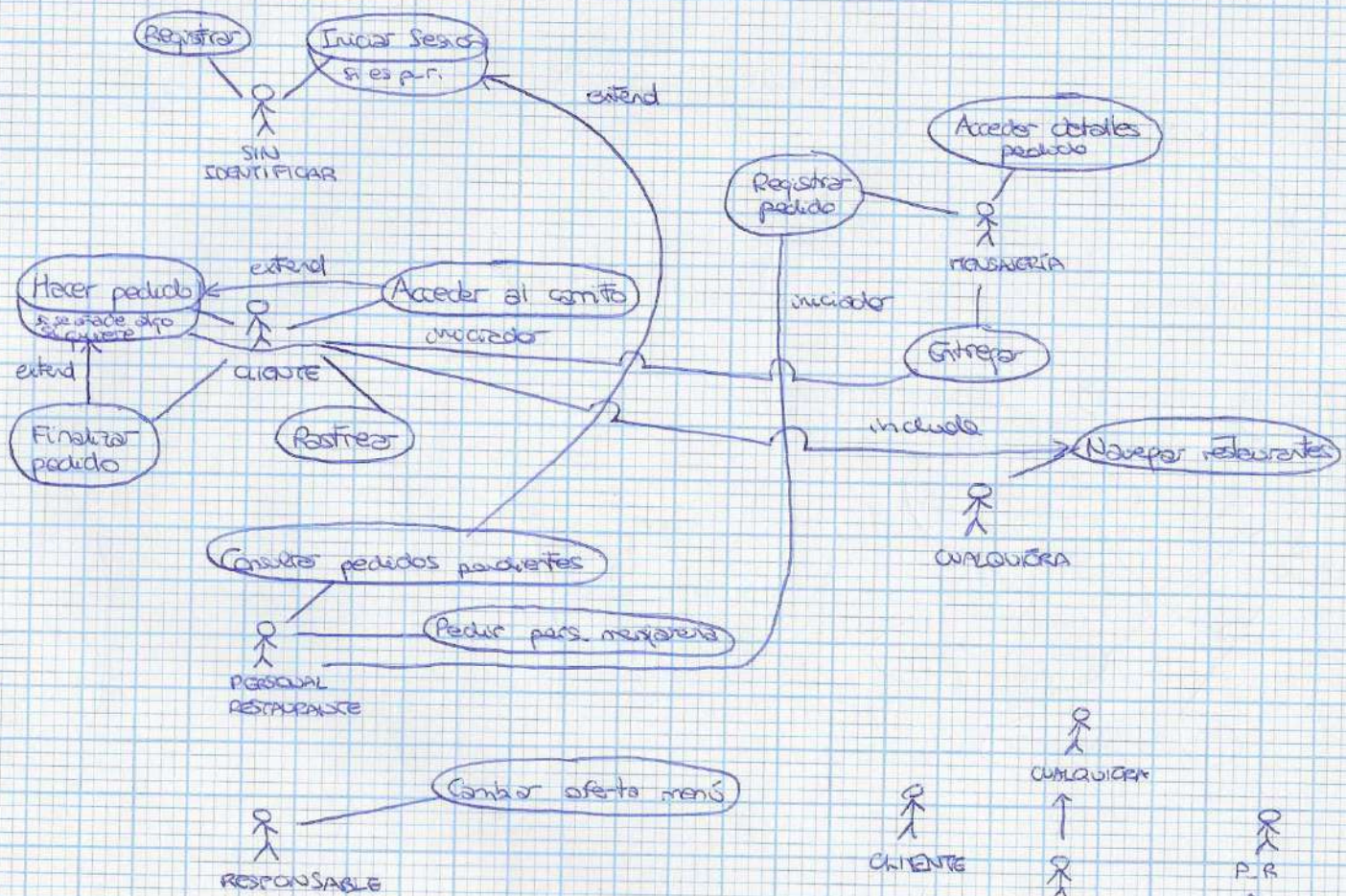
2.3. pagarVisa (num, cad, nom, 1000)

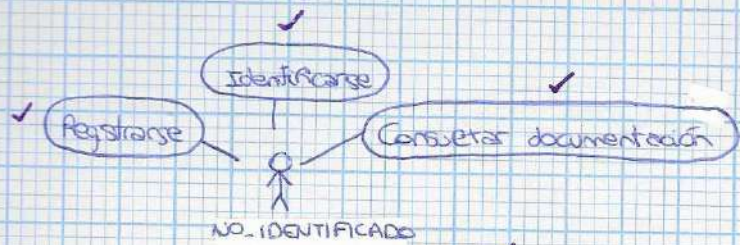
2.4.- añadirPuntos (elCliente, 10) : void

2.4.1.- addPtosAConsumer (10) : void

Gestor Clientes
todosClientes:
listaClientes

Discoteca
- entrar.....





NO IDENTIFICADO

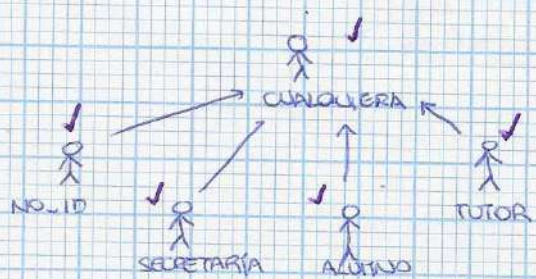
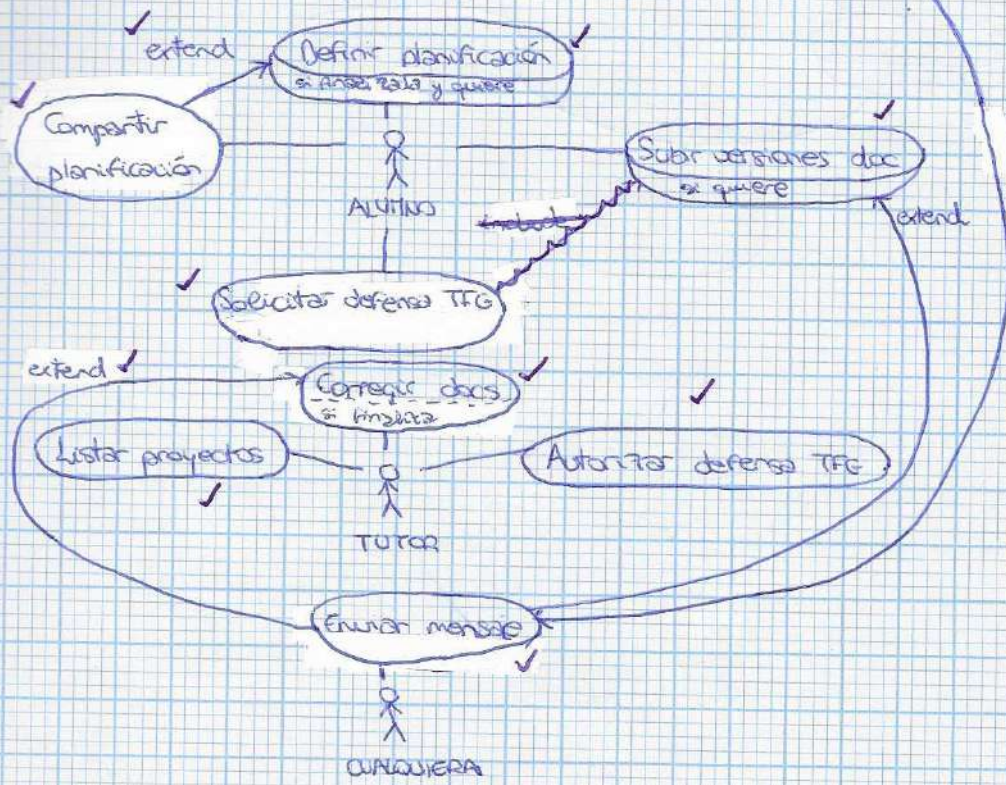


SECRETARIA

ALUMNO

TUTOR

CUALQUIERA



Identificarse



NO IDENTIFICADO

Gestionar sabores
si quiere

extend

Añadir sabor
si quiere

Poner en marcha
producción



GERENTE
entend

Actualizar cant.
caramelos



ENCARGADO

Analizar pedidos



ALMACÉN

[Redacted]



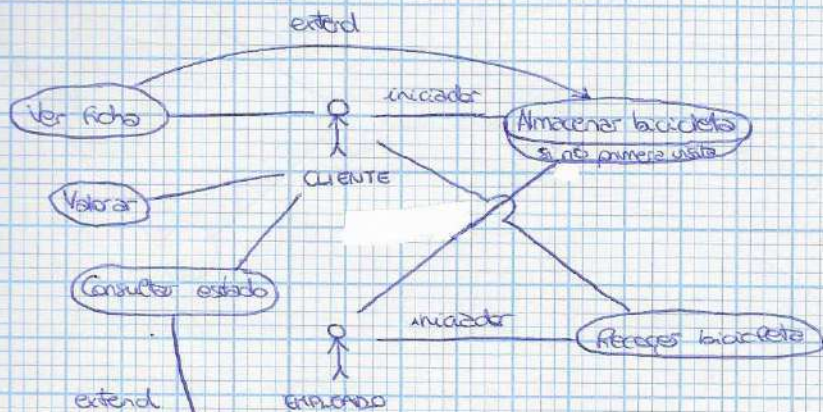
CLIENTE

Actualizar cant. caramelos

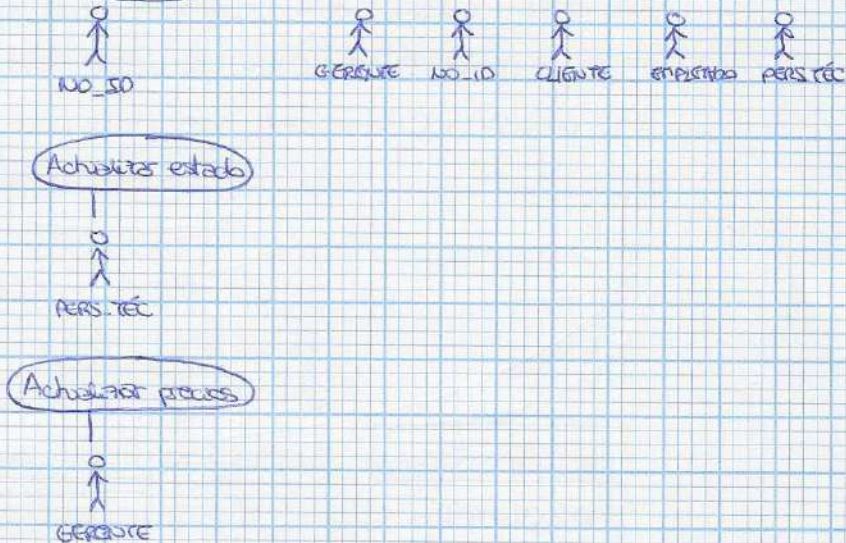


CUALQUIERA

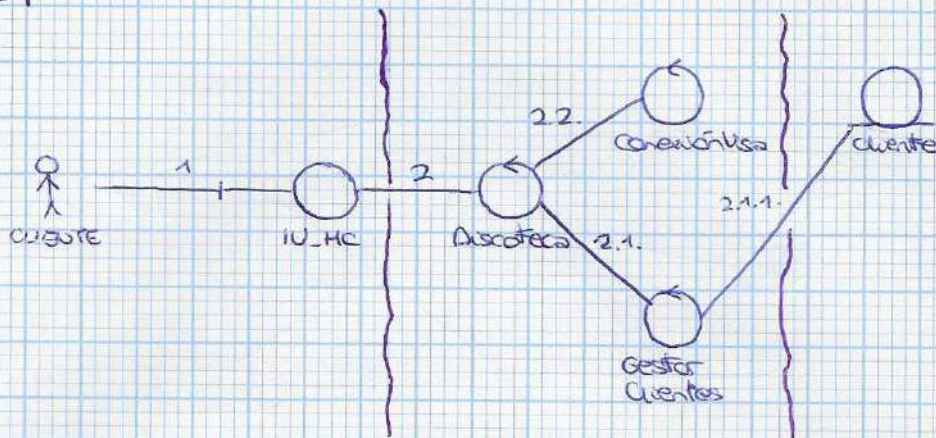
22-



Jerarquía



EJ. 1



1. Proporciona datos

2. crearCliente (nom, dir, tefn, cad):

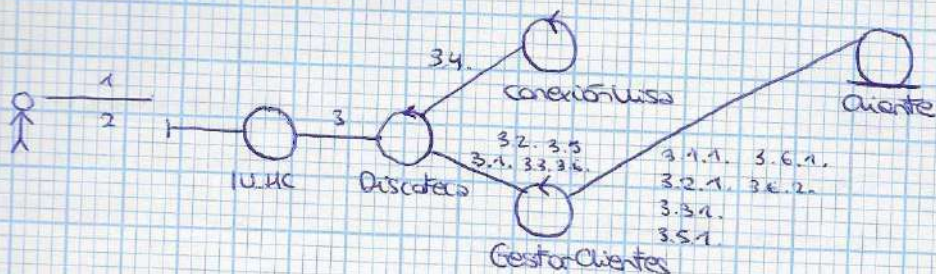
2.1. añadir Clientes (nom, dir, tefn, cad, numv):

2.1.1. new Cliente (nom, dir, tefn, cad, numv):

2.2. pagarVisa (numv, cad, nom, gso)

Gestor Clientes
todos Clientes: lista Clientes

DiscoFeca
- crear Cliente (...)



1. Introducir numSoc
2. Introducir numCon y numSin
3. consumir (numSoc, numCon, numSin)
 - 3.1. elCliente = buscarCliente (numSoc) : Cliente
 - 3.1.1. getCodigoCliente () : int
 - 3.2. obtenerPtos (elCliente) : int
 - 3.2.1. getPtosAConsumir () : int
 - 3.3. obtenerDatosUsa (elCliente) : JSon
 - 3.3.1. obtenerDatosUsa () : JSon
34. pagarUsa (numv, cod, nombre, eco)
- 3.5. esPreferente (elCliente) : bool
 - 3.5.1. esClientePreferente () : bool
- 3.6. actualizarInfo (elCliente, ptosConsum, ptosRes)
 - 3.6.1. addTotalPtosConsum (ptosConsum)
 - 3.6.2. setPtosAConsumir (ptosRes)

Película

- Título
- Nivel de Bechdel
- SusAct : Lista Actuación

Interprete

- nombre y apellidos

Personaje

- Nombre
- Sexo

Actividades

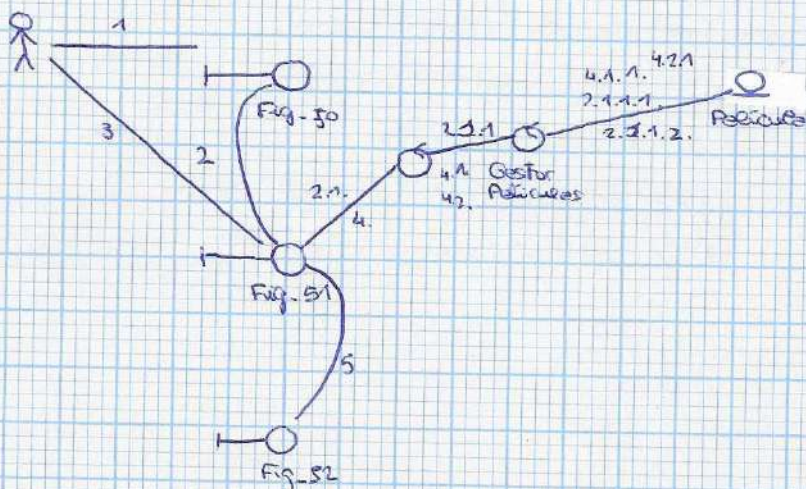
- Pers: Personaje
- Quien: Interprete

Diálogo

- Cuando: Date
- Hablador: Personaje
- el otro: Personaje
- Aonde: Película

Gestor Películas

todas : Lista Película



1. Pulsar botón Test de Bechdel

2. new Fig_51()

2.1. `ExistirPelículasasNivel(): JSon { String, ... }`

2.1.1. `ExistirPelículasasNivel(): JSon`

2.1.1.1. `tieneNivel(): bool`

[S: F] 2.1.1.2. `getTitulo(): String`

3. Selecciona Peticiones y pulsa "Comprobar Nivel"

4. comprobarNivel (titulos)

4.1. encapsular Peticiones ("Títulos"): Peticiones

4.1.1. getTitulo(): string

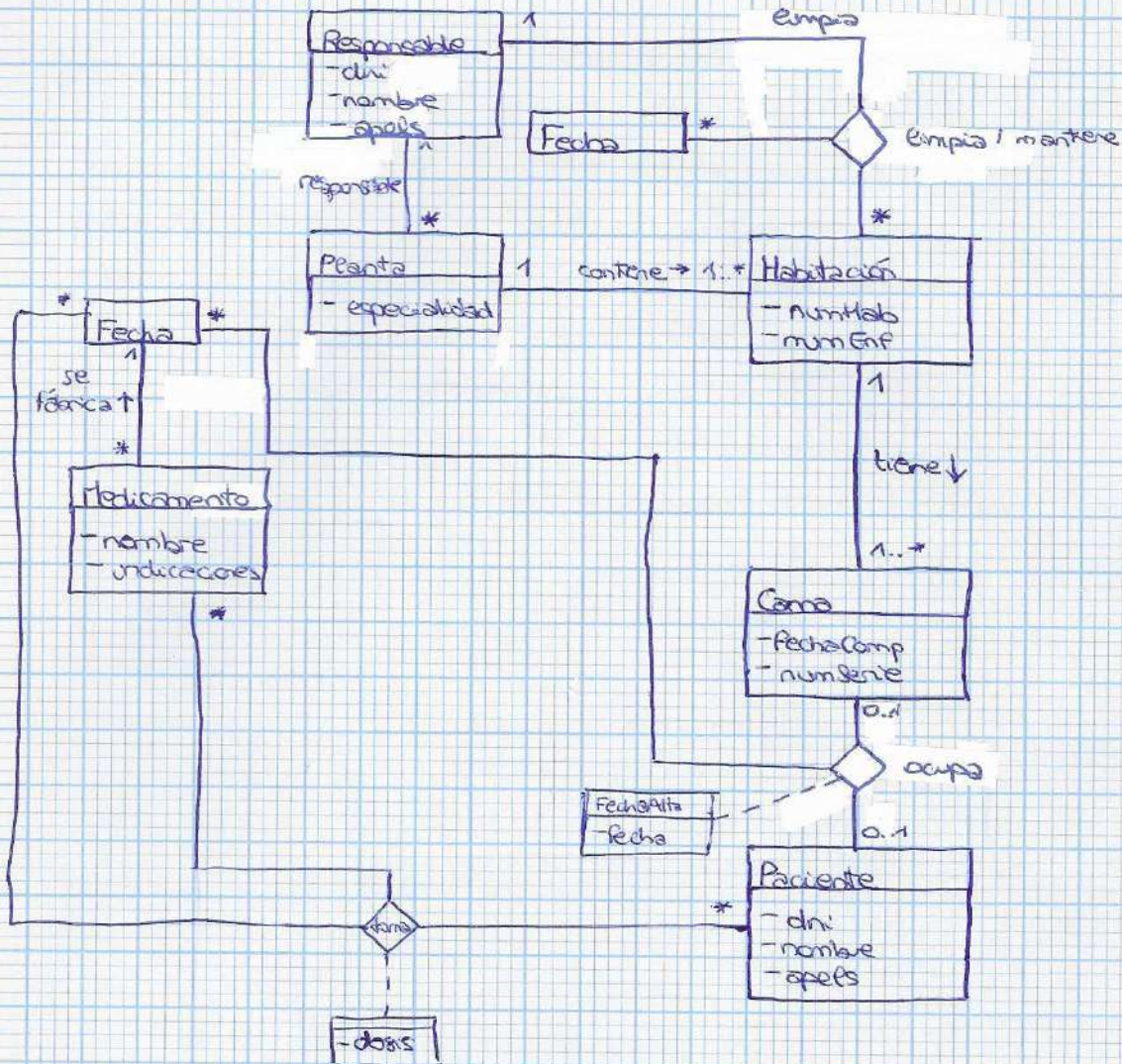
4.2. obtNumPerSfem (@Peticiones): int

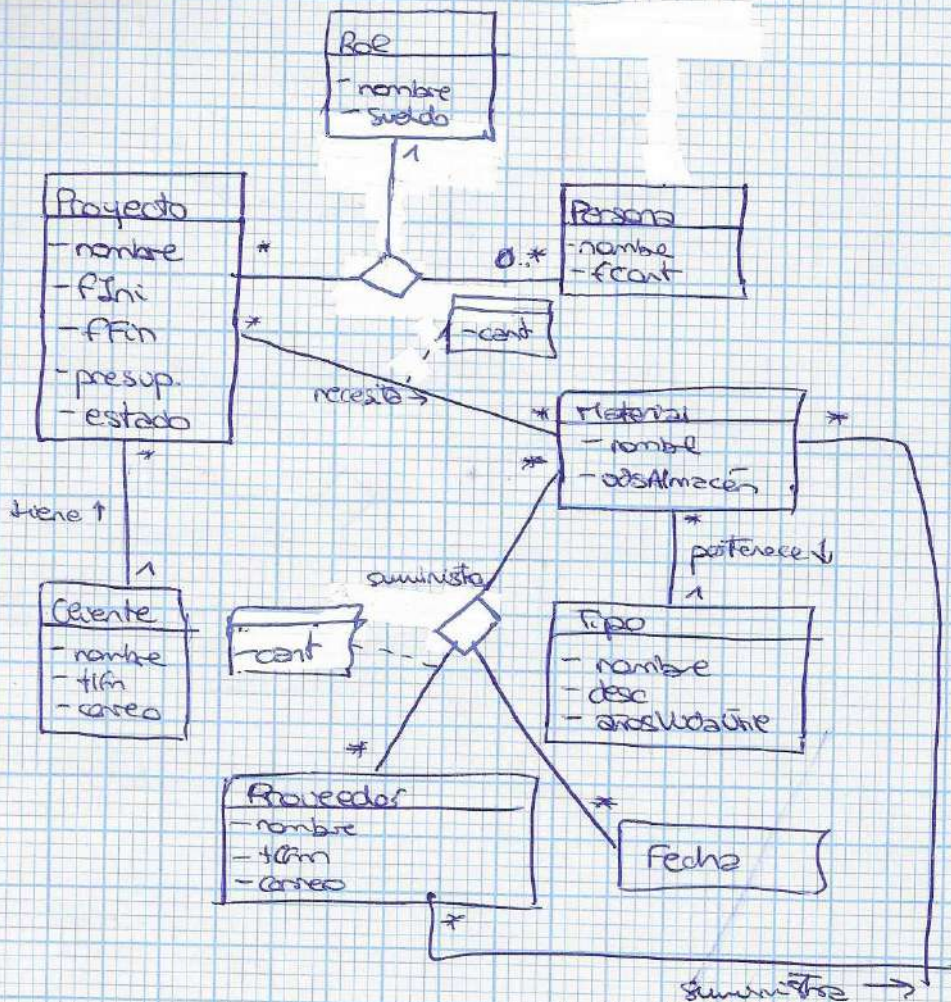
4.2.1. obtenerPerSfem(): int

4.2.1.1. obtenerSfem(): bool

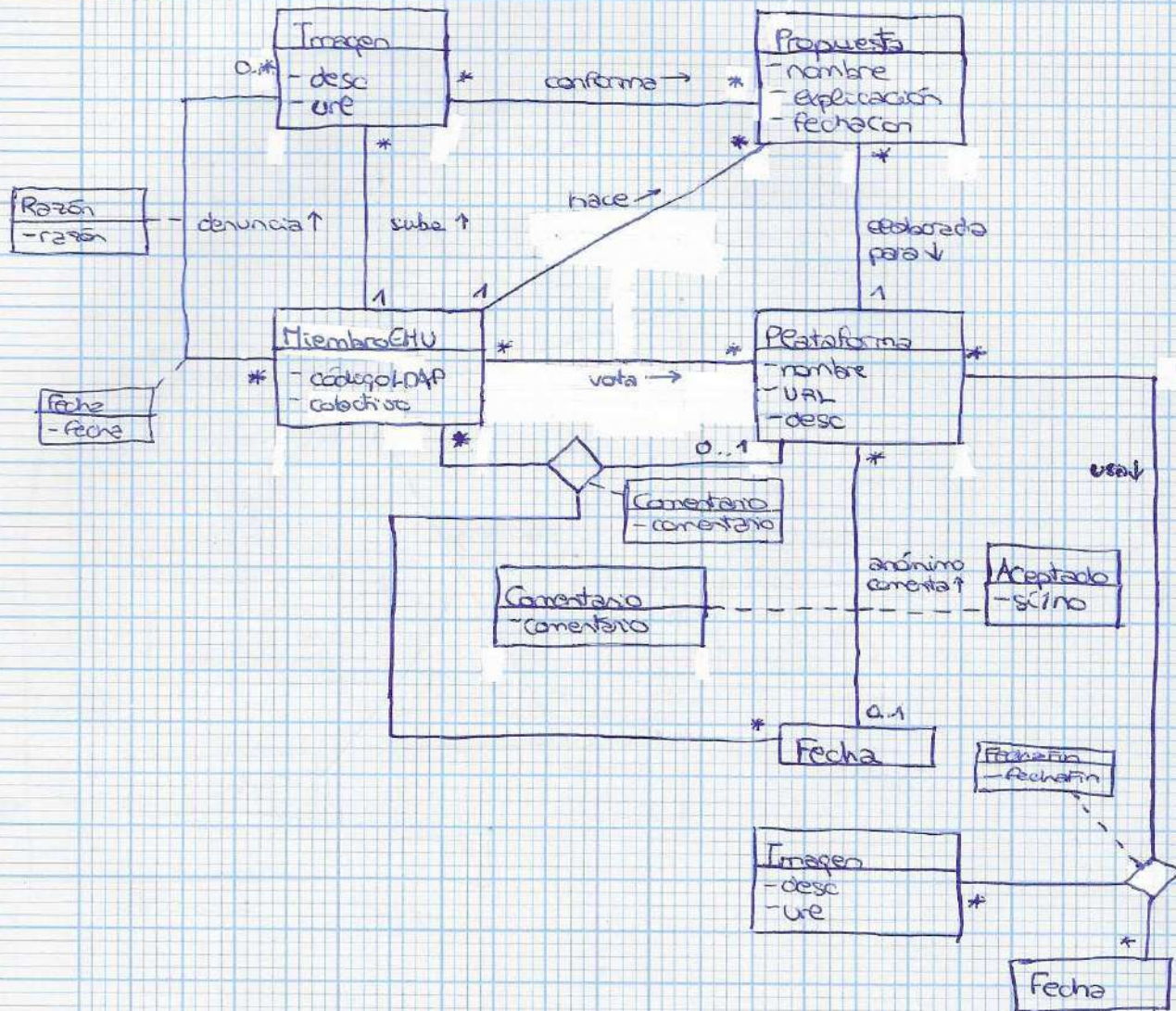
4.2.1.1.1. getServo(): string

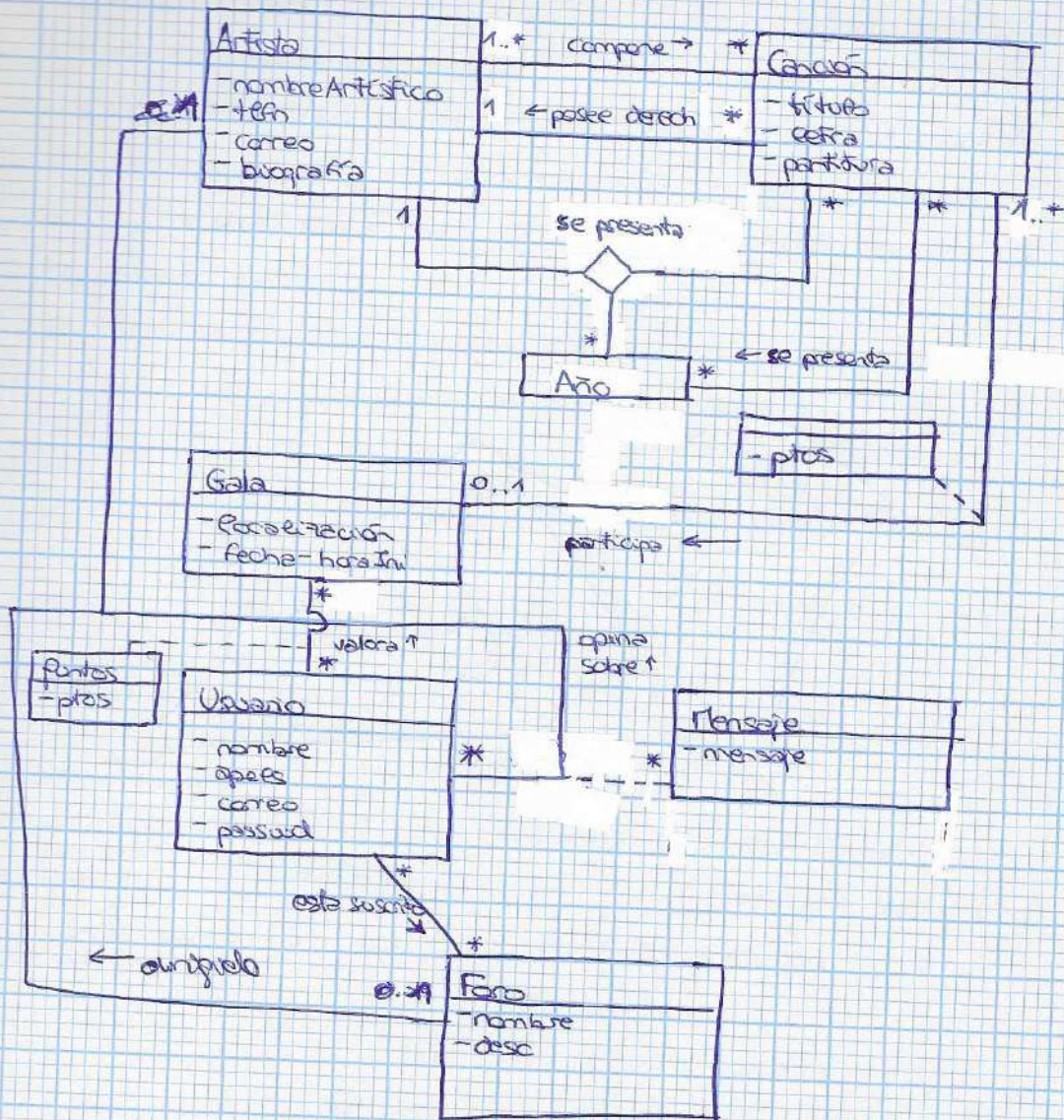
5. new Fig S2 (chibao)





49



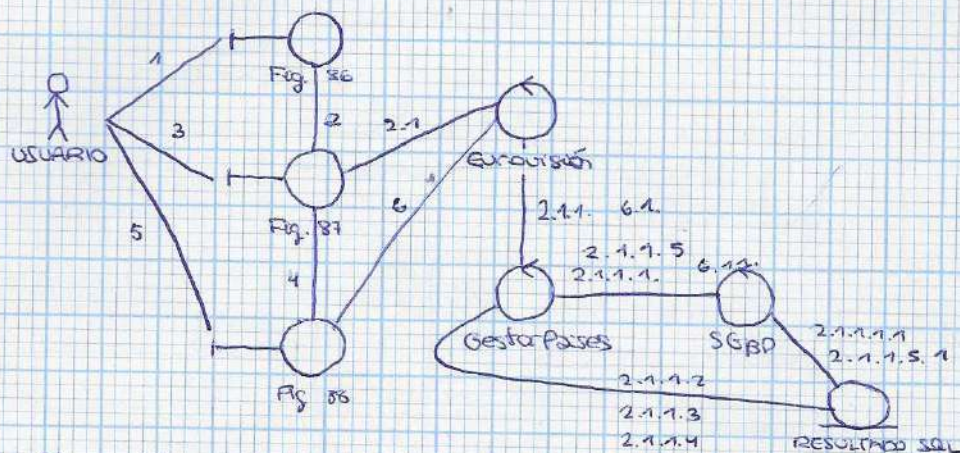


País (Nombre, Bandera, TV)

Artista (Nombre Artístico, Num miembros, codArtista)

Participación (Nombre, Año, Asistirá?)

Presentación (CodArtista, Nombre, Año, título, Puntos)



1- Pasa "Actualizar Información"

2- New Figura87()

2.1.- obtenerInfoIncompletos(): JSON, { nombre: string,
 años: int,
 banderas: Image,
 ... }
 }
 }
 }

2.1.1.- obtenerInfoIni(): JSON = JSON

2.1.1.1.- execSQL(sql): ResultadoSQL

2.1.1.1.1.- newResultadoSQL()

2.1.1.2.- next()

2.1.1.3.- getString("Nombre"): String

2.1.1.4.- getImage("bandera"): Image

2.1.1.5.- execSQL(sql2): ResultadoSQL

Eurovision

GestorPaíses

sql2 = SELECT nombre,
 bandera FROM País
 NATURAL JOIN Participación
 WHERE Asistirá? <= true
 AND Año = % año% AND
 nombre NOT IN C SELECT
 nombre FROM Representación
 WHERE Año = % año%

2.1.1.5.1. - new ResultadoGOLC()

2.1.1.6. - next()

2.1.1.7. - getString(3 "Nombre") : string

2.1.1.8. - getImage("bandera"): Image

3 - Seleccionar País y pulsar "Votar"

4 - new Fig 5r (*nombre, bandera)

5 - Indicar asistencia

6 - Almacenar Asistencia (nombre, año, respuesta)

6.1. - almacenar Asistencia (nombre, año, respuesta)

6.1.1. - execSQL (sql₃)

sql₃ = INSERT INTO

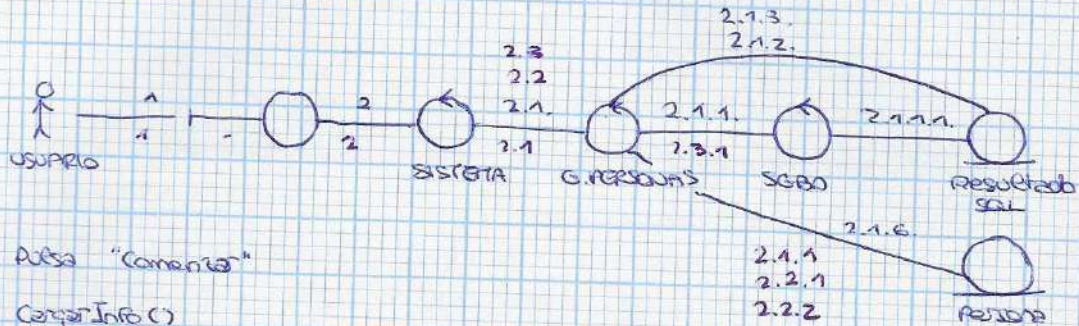
Participacion (nombre, año,
asistirá?) VALUES (%nombre%,
%año%, %respuesta%)

Persona (DNI, nombre, edad)

Persona
- DNI: String
- nombre: String
- edad: int

Gestor Persona
- todas: List<Pers>

sqe₁ = SELECT * FROM Persona



1.- Pulsar "Comenzar"

2.- CargarInfo()

2.1.- cargarPersonas()

2.1.1.- `execSQL(sqe1)`: ResultadoSQL

2.1.1.1.- `new ResultadoSQL()`

2.1.2.- `next()`: `bool`

2.1.3.- `getDatos()`: `JSON` = { nombre: String, DNI: String, edad: int }

2.1.4.- `new Persona(DNI, nombre, edad)`

1.- Introducir DNI y nueva edad

2.- actualizarEdad(DNI, edad)

2.1.- `BuscarPersona(DNI)`: Persona

2.1.1.- `getDNI()`: String

2.2.- `actualizarEdad(BuscarPersona, edad)`: int

2.2.1.- `getEdad(edad)`

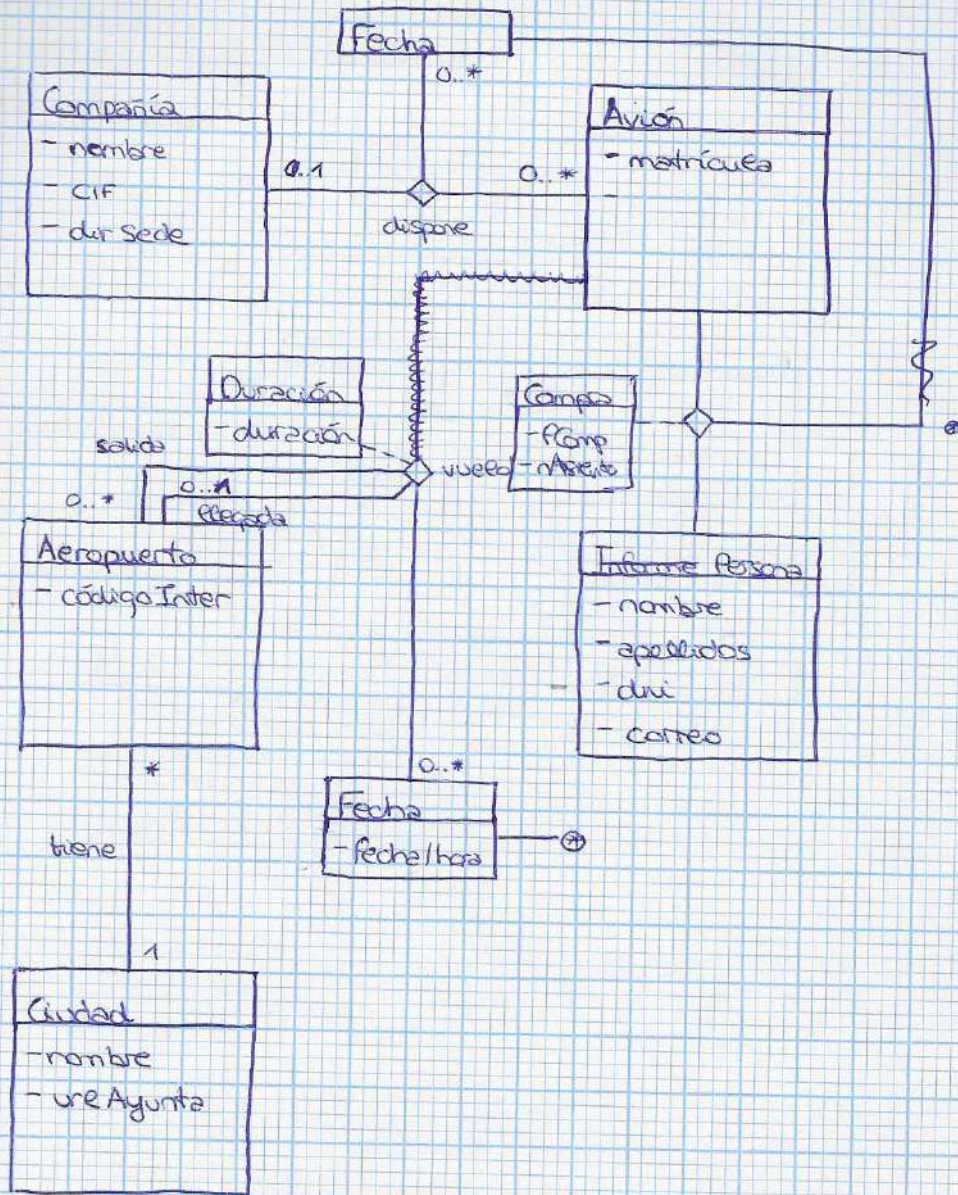
2.2.2.- `getCodP()`: int

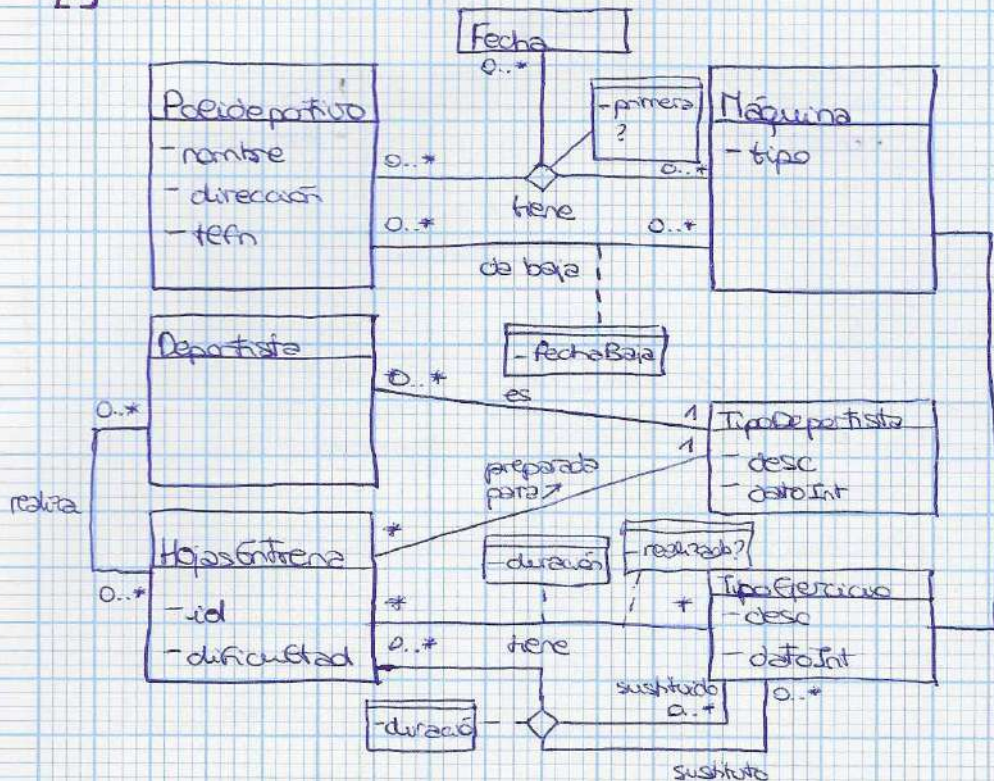
2.3.- `actualizarEdad(codP, edad)`

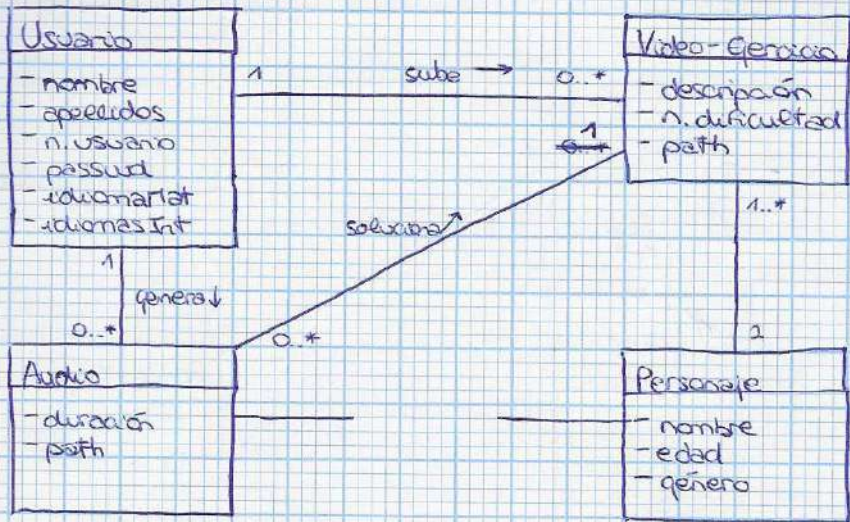
2.3.1.- `execSQL(sqe2)`

sqe₂ = UPDATE FROM

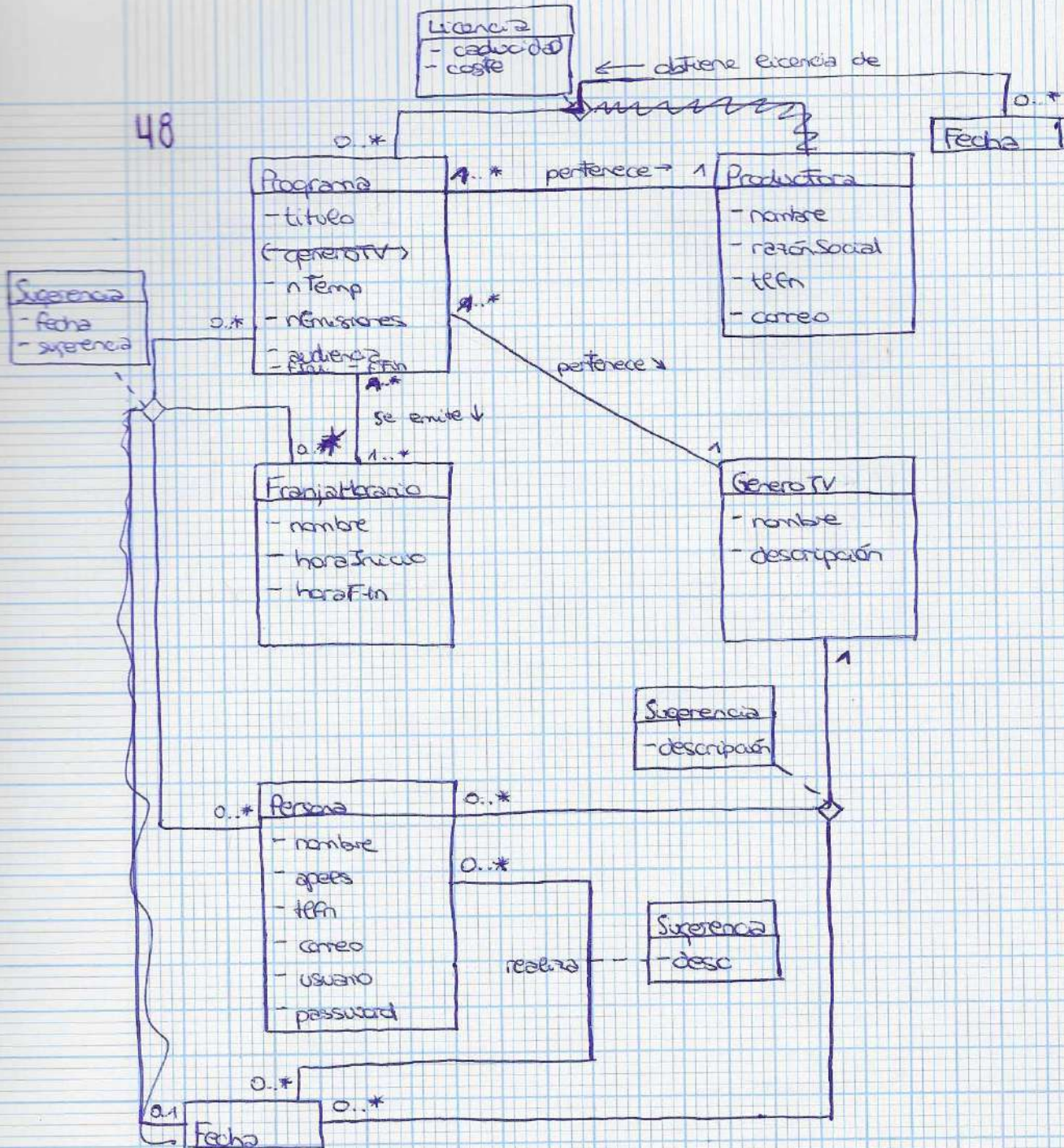
Persona SET edad = %edad%
WHERE codP = %codP%



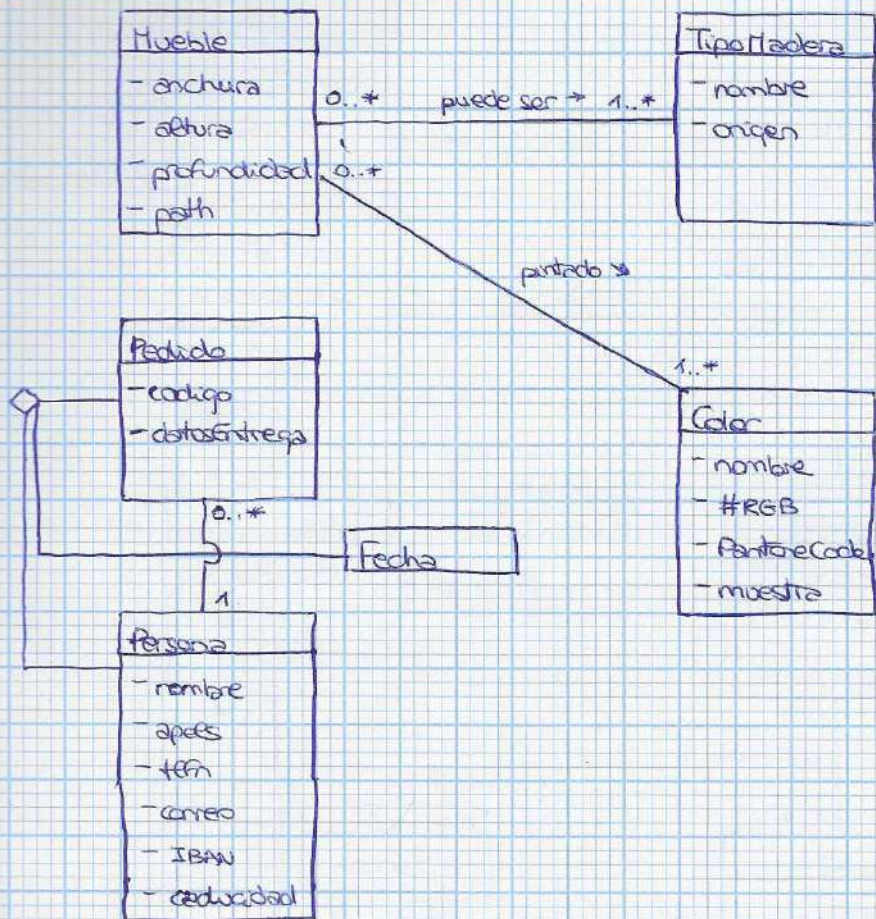


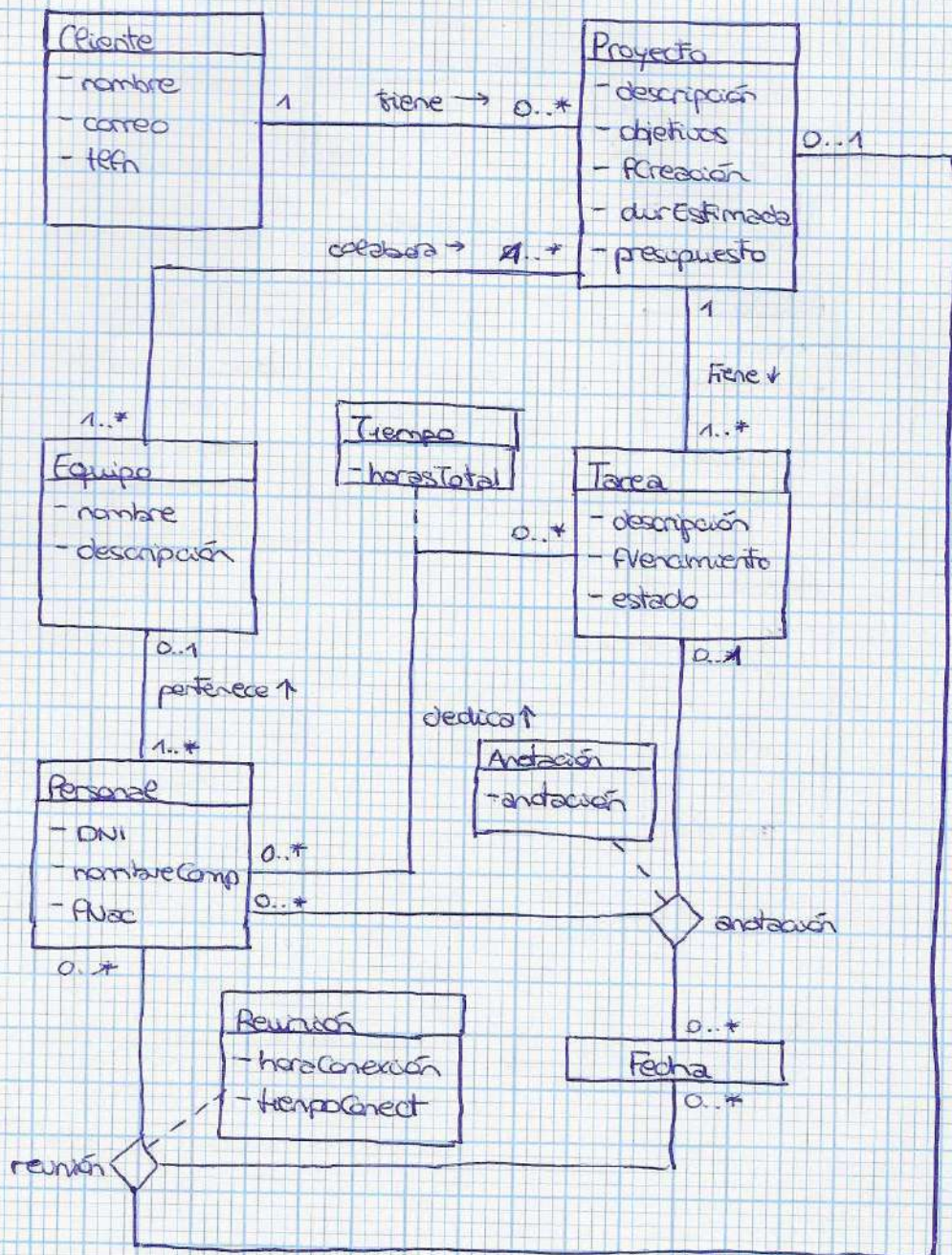


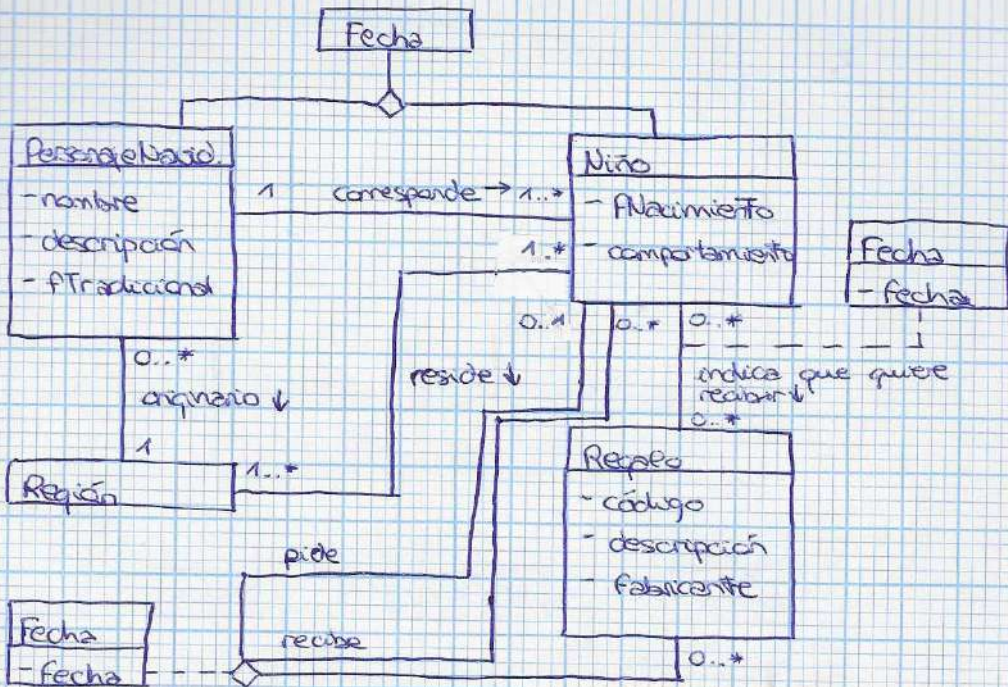
48

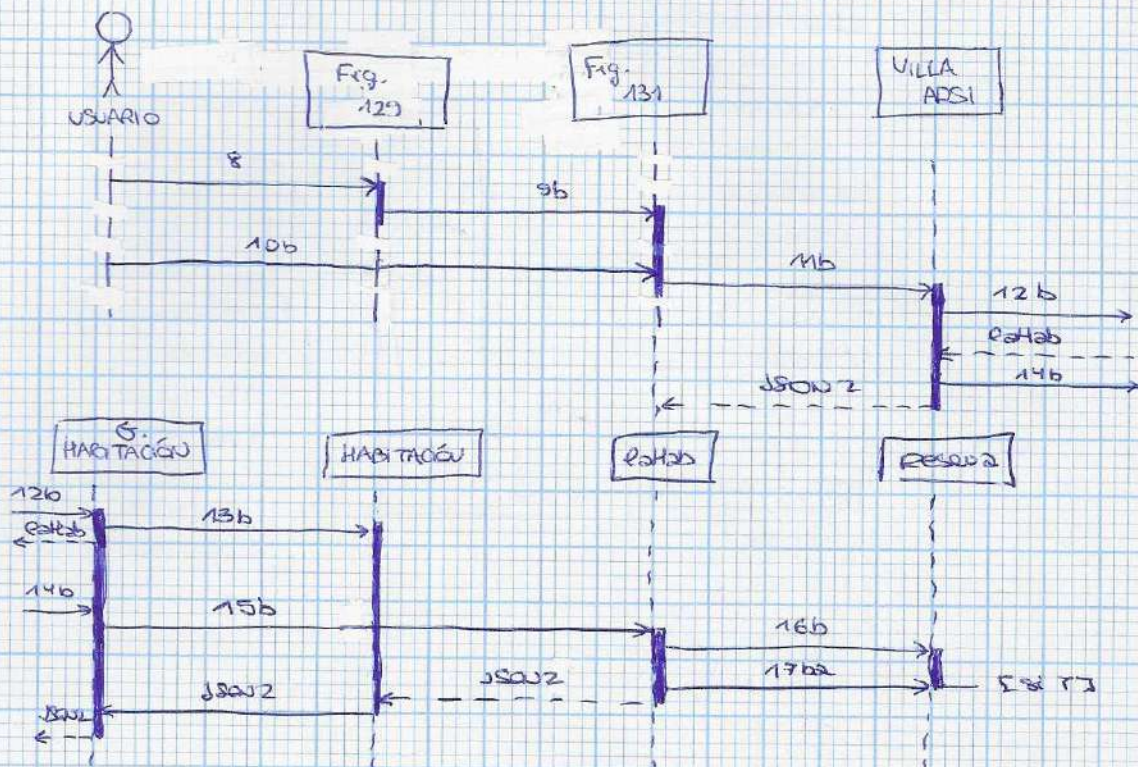


47









8- Pulsar "algo"

[si es una habitación]

9b- new Fig131 (num)

10b- Seleccionar un día

11b- obtenerInfoReserva (num, día): JSOU2

12b- Pstado.buscarHabitación (num): Habitación

13b- es? (num): bool

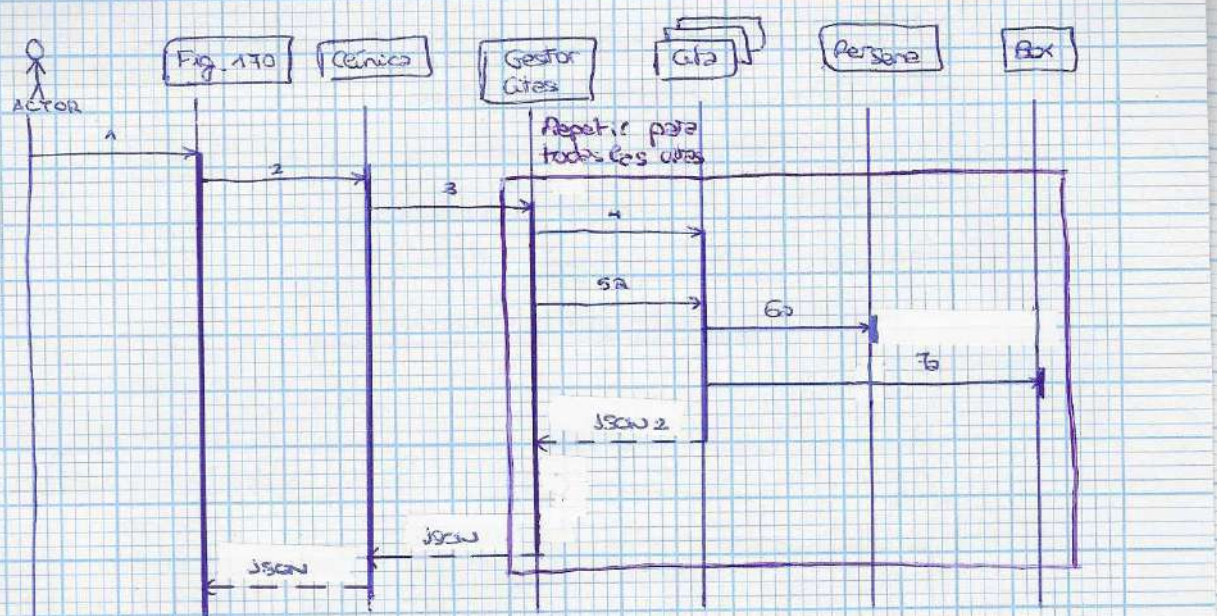
14b- obtenerInfoReserva (numHabit, día): JSOU2

15b- obtenerInfoReserva (día): JSOU2

16b- esActiva? (día): bool

[si T]

17b- getInf() : JSOU2

Cita

- horaInicio: Date
- horaFin: Date
- box: Box
- persona: Persona

Box

- numBox: int

Personal

- nombreApels: String
- codPers: int
- usuario: String
- contraseña: String

Clínica ADSITratamiento

- tratamiento: String
- fechaIni: Date
- personal: Personal

Gestor Citas

- listaCitas: Collection
< Cita >

Persona

- dni: String
- Nombre: String
- FechaNac: String
- Foto: Foto
- listaTratamientos: Collection < Tratamiento >

Foto

- fecha: Date
- path: String

Accesos (DUI, FechaHora, CodAsignatura)

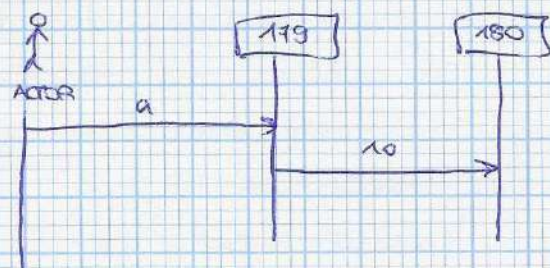
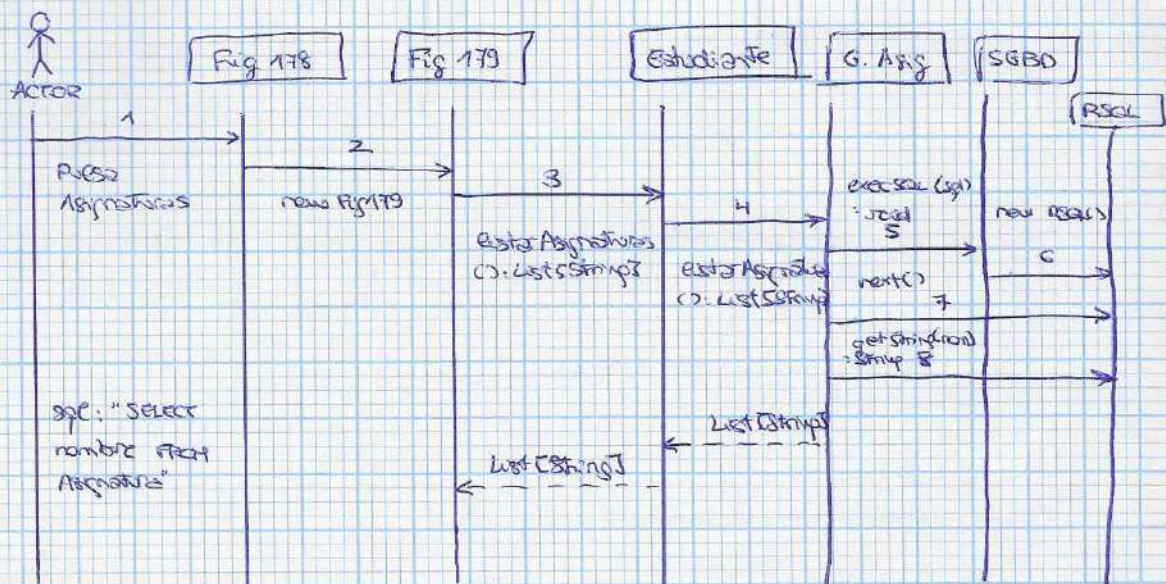
Asignatura (CodAsignatura, nombre)

Tarea (CodTarea, esc, AlumEntrega, CodAsignatura)

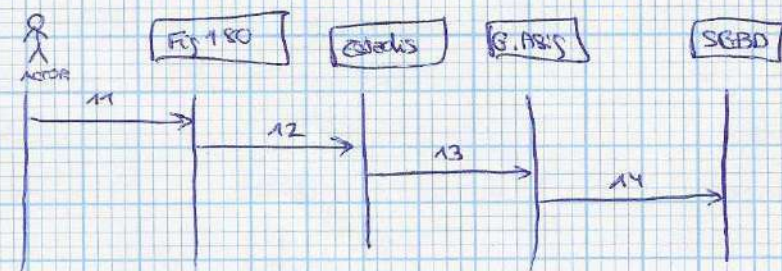
Estudiante (DUI, Nombre, Apellidos, FNac)

Entrega (CodTarea, DUIest, FNEntrega)

Matriculas (CodAsignatura, DUIestudiante)



9- Seleccionar una Asignatura
10- new Fig180()



11.- se puede borrar
12.- borrarAsignatura (Asignatura)
13.- borrarAsignatura (esp)
14.- ejecutar ("DELETE + FROM Asignatura WHERE nombre = %s" % cu cascode")

Paciente

- nomApels: String
- edad: int
- nss: int
- genero: String
- antecedentes: String

Gestor Pacientes

- listaPacientes: Collection<Paciente>

Cama

- numCama: int
- numSere: int

Habitación

- numHabitacion: int
- tamaño: int
- listaCama: Collection<Cama>

Planta

- numPlanta: int
- plantaCuid: int
- listaHab: Collection<Habitacion>

Ingreso

- fechaAlta: Date
- positivoCovid: boolean
- fechaHora: DateTime

Ejercicio 29

Diagrama de Clases

Habitación
- códigoHab: int
- nHab: int
- estadoHab: List[Cama]

Cama
- código: int
- nCama: int

Paciente
- DNI: String
- nomApels: String

Suministro
- dosis: int
- suministrado?: boolean
- razónNoSum: boolean

Medicamento
- códigoMed: int
- nombre: String

Personal Sanitario
- DNI: String
- Usuario: String
- contraseña: String

Seudónimos

Gestor Personal Sanitario
- estadoPS: List[PS]

Base de Datos

Habitación	
PK	códigoHab
	nHabitación

Pacients	
pk	DNI
	nomApels

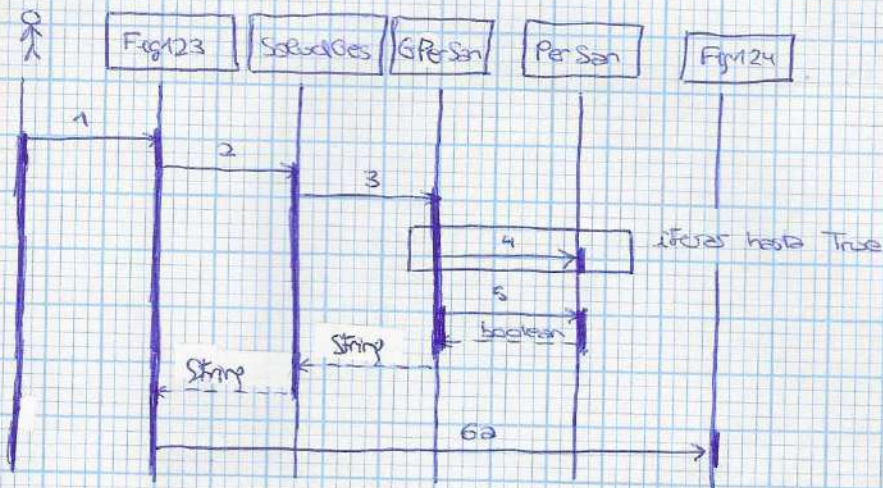
Medicamentos	
PK	codificado nom/laye

Estancia	
Pk1, Pk2	codCama
Pk2, Pk3	DNI Pac
Pk3	Fecha

Cama	
PK	código
	numCama
PK1	códigoHab

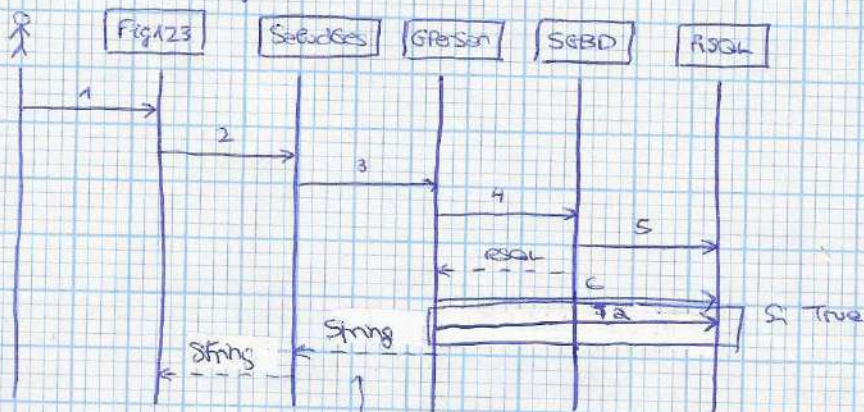
Suministro	
DNI, FIC	DNI, FIC
PK1, PK2	DNI, PS
PK3, FIC	codMed
DNI, FIC	fecha
	dosis

Personal Sanitario	
PK	DNI
	Usuario
	Contraseña



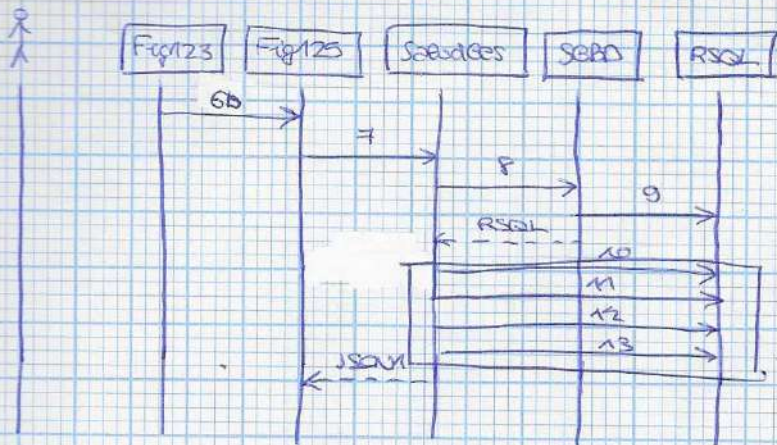
1. El personal sanitario introduce su nombre y contraseña y pulsa "Aceptar"
2. `incSesionPS (pNombre: String, pPassword: String): boolean`
3. `incSesionPS (pNombre: String, pPassword: String): boolean`
4. `esPS (pNombre: String): boolean`
5. `verificarPassword (pPassword: String): boolean`
(if False)

6a. new Fig124()



Si el String es NULL = False, si no True

1. "
2. `incSesionPS (pNombre: String, pPassword: String): boolean`
3. `incSesionPS (pNombre: String, pPassword: String): boolean`
4. `EXEC SQL ("SELECT DNI FROM PS WHERE usuario = %s pNombre% AND contra = %s pPassword")`
5. `new RSQAL()`
6. `ret(c): boolean`
- 7a. `getStrinp ("DNI"): String`



(if True)

6b. new Fig125(poni: String).

7. getPatientes(poni: String): JSQVI

8. executeSQL("SELECT p.nomApels, c.codigo, h.codigoHab FROM ((Somministra AS s NATURAL JOIN estancia AS e) INNER JOIN Paciente AS p ON s.DNIPar = p.DNI) INNER JOIN cama AS c ON e.codigoCama = c.codigo) NATURAL JOIN Habitacion AS h WHERE s.DNIPS = %poni% AND

9. new RSQL()

10. next(): boolean

11. getString("nomApels"): String

12. getInt("codigo"): int

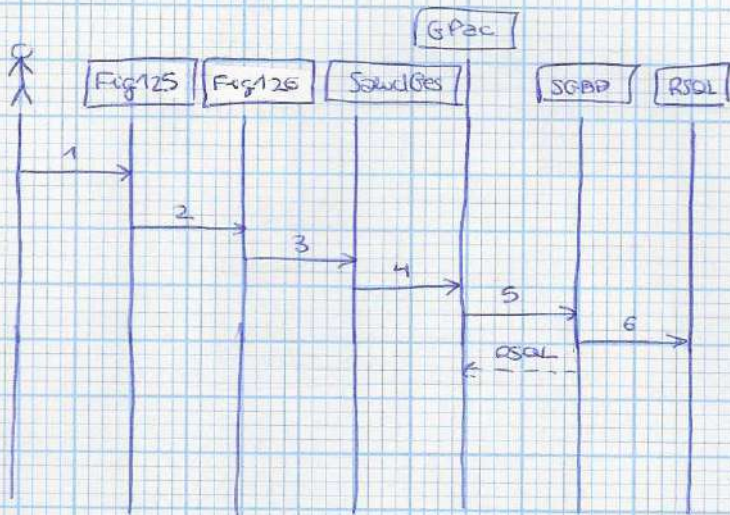
13. getInt("codHab"): int

JSQVI: FE

```

{
  nomApels: String, codigoCama: int, codigoHab: int
}
}

```

1. Pasa "atender" correspondiente a un paciente
2. new Fig126(pNomApels : String)
3. getMedicamentos(pNomApels : String)
- 4.

Autor	
	nombreAu apellidos seudonimo añoPub editorial añoFuente
PK	

Libro	
	títuloLibro numPaginas añoPubl editorial seudAutor
FK1	

Cliente	
	nombreCliente apellidos numVisa fechaCadVisa tfeñ
AK	

Opinión	
	valoración PK, FK títuloLibro FK, FK numCliente

Compra	
	numUnidades servido?
FK, FK	numVisaLib
FK, FK	títuloLibro
FK3	fecha

Premios	
PK	nombrePremio dataciónEcono

Premiación	
FK, FK	nombrePre
FK, FK	títuloLibroPre
FK3	fechaRemia

Colección	
PK	nombreCole descripción

Referencia	
FK, FK	títuloLibro
FK, FK	nombreColecc

Premios	
-	nombrePremios: String
-	dataciónEcono: int

Colección	
-	Nombre: String
-	descripción: String

Librerías	
<<constructor>>- Librerías()	

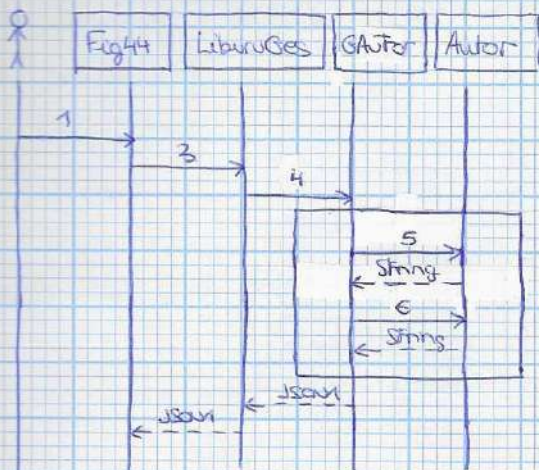
Autor	
-	nombreAutor: String
-	apellidosAutor: String
-	seudonimo: String
-	añoPub: int
-	añoFuente: int
-	listaLibros: List<Libro>
<<constructor>>+ Autor(...)	

Libro	
-	títuloLibro: String
-	numPaginas: int
-	añoPubl: Date
-	editorial: String

Cliente	
-	nombreCliente: String
-	apellidos: String
-	numVisa: int
-	fechaCadVisa: Date
-	tfeñ: int

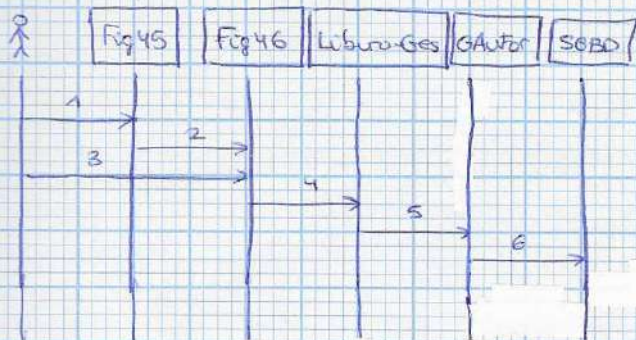
Opinión	
-	valoración: String

Compra	
-	numUnidades: int
-	servido?: boolean



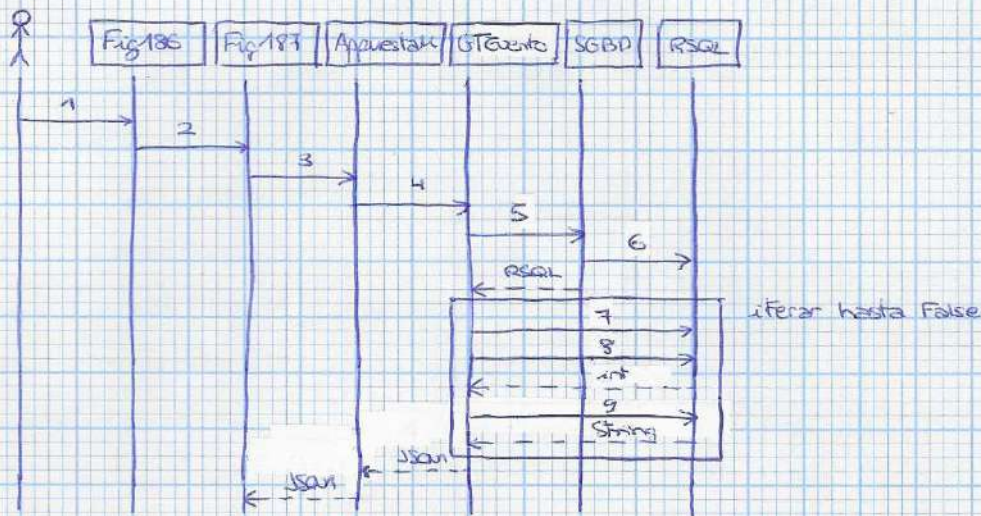
1. El Usuario pulsa "Gestionar Autores"
2. new Fig45()
3. getAutores(): JSON
4. getAutores(): JSON
5. getString("Nombre"): String
6. getString("Apellidos"): String

JSON: {{"Nombre: String, Apellidos: String, ... } }



1. El usuario pulsa "Añadir Autor"
2. new Fig46()
3. Introduce los datos y pulsa "Aceptar"
4. addAutor(...): void
5. addAutor(...): void
6. executeSQL(sql, String): void

sql: "INSERT INTO Autor VALUES(...)"



1. Se pulsa "Apostar"
2. new Fig187()
3. getTipoEvento(): JSon
4. getTipoEvento(): JSon
5. executeSQL("SELECT * FROM TipoEvento"): RSQL
6. new RSQL()
7. next(): boolean
8. getEnt("codTipoEvento"): int
9. getStrng("descripcion"): String

JSon: {

{ codTipoEvento: int, descripcion: String }

1, ...

}

Evento	
PK	codEvento
	descripcion
	fechaIni: PA
	fechaFin: PA
	importetIn
FK1	codTipoEvento

TipoEvento	
PK	codTipoEvento
	descripcion

Apuesta	
FK1, FK2	DNI: Persona
FK3, FK4	codOpcion
FK5, FK6	fechaHora
	cantidadApos
	cantidadACobrar

Opcion	
PK	codOpcion
	descripcion
	probabilidad
FK1	codEvento

Persona	
PK	DNI
	nombreApels
	usuario
	contraseña

Appuestak	
«constructor» + Appuestak()	
+ getTiposEvento(): JSau1	
+ getEventoTipo(codTipoEvento: int): JSau2	
+ getOpcionesEve(codEvento: int): JSau3	

GEvento	
- listEvento: List<Evento>	
«constructor» + GEvento()	

TipoEvento	
- codTipoEvento: int	
- descripcion: String	
- listEventos: List<Evento>	

Evento	
- codEvento: int	
- descripcion: String	
- fechaIni: PA	
- fechaFin: PA	
- importetIn: int	
- listOpciones: List<Opcion>	

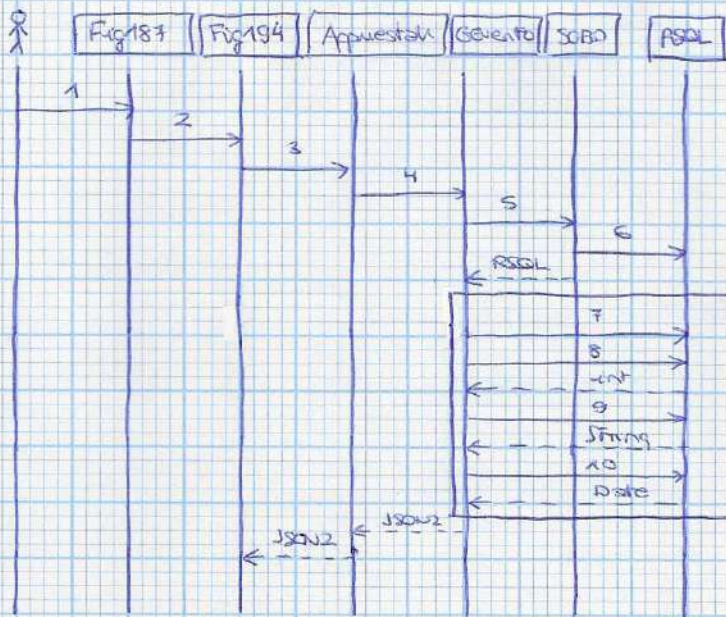
SQL	
«constructor» + SQL()	
+ execSQL(sqe: String): SQL	
+ executeSQL(sqe: String): void	

RSOL	
«constructor» + RSOL()	
+ next(): boolean	
+ getString(campo: String): String	
+ getInt(campo: String): String	
+ getDate(campo: String): Date	
+ getDouble(campo: String): Double	

GEvento	
- listEvento: List<Evento>	
«constructor» + GEvento()	
+ getEventoTipo(codTipoEvento: int): JSau2	

GOpciones	
- listOpciones: List<Opcion>	
«constructor» + GOpciones()	
+ getOpcionesEve(codEvento: int): JSau3	

Opcion	
- codOpcion: int	
- descripcion: String	
- probabilidad: Double	

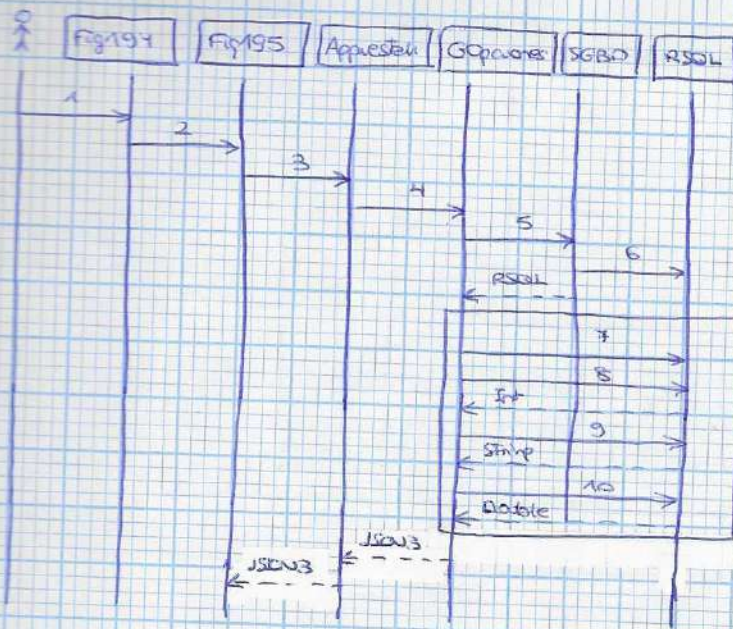


1. Se pueba el tipo de evento sobre el que se quiere apostar
2. `new Fig194 ( codTipoEvento : int, descripcionEvento : String )`
3. `getEventoTipo ( codTipoEvento : int ) : JSOL2`
4. `getEventoTipo ( codTipoEvento : int ) : JSOL2`
5. `executeSQL ( " SELECT codEvento, descripcion, fechaFinPA FROM Evento WHERE codTipoEvento = % codTipoEvento % AND fechaIniPA <= % now % AND fechaFinPA >= % now % " ) : RSOL`
6. `new RSOLC ()`
7. `next () : boolean`
8. `getInt ( " codEvento " ) : int`
9. `getString ( " descripcion " ) : String`
10. `getDate ( " fechaIniPA " ) : Date`

JSOL2 : {

{ codEvento : int, descripcion : String, fechaIniPA : Date }
 , ... }

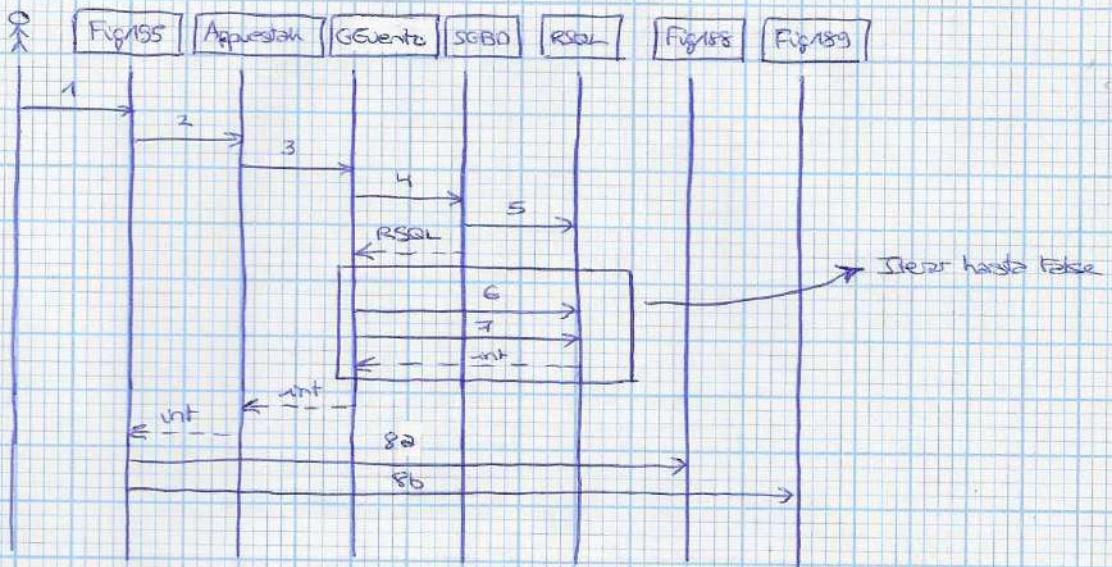
}



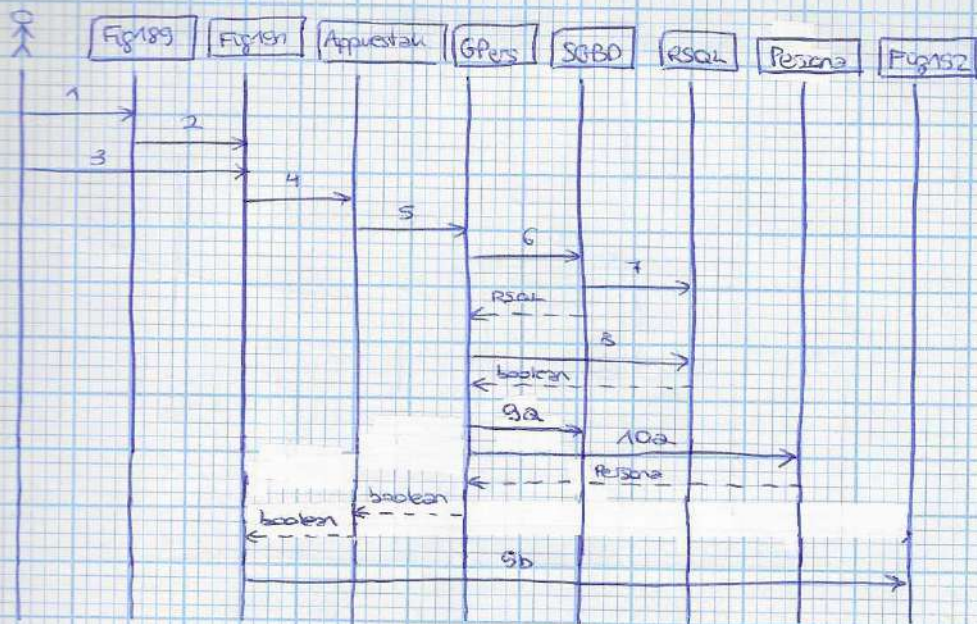
1. Se pulsa sobre un evento
2. new Fig195 (codEvento : int, descripcion : String)
3. getOpcionesEve (codEvento : int) : JSOL3
4. getOpcionesEve (codEvento : int) : JSOL3
5. executeSQL ("SELECT codopcion, descripcion, probabilidad FROM opcion WHERE codEvento = %codEvento%") : RSOL
6. new RSOLC()
7. next() : boolean
8. getInt ("codopcion") : int
9. getString ("descripcion") : String
10. getDouble ("probabilidad") : Double

JSOL3 = {}

{ codopcion : int, descripcion : String, probabilidad : Double
 ... }



1. Se elige una opción de apuesta, se introduce la cantidad a apostar y se pulsa "Apostar"
2. `getLin ( codEvento : String ) : int`
3. `getLin ( codEvento : String ) : int`
4. `executSQL ( "SELECT importelin FROM evento WHERE codEvento = '%codEvento%'" ) : RSOL`
5. `new RSOL()`
6. `next() : boolean`
7. `optInt ( "importelin" ) : int`
- (Si el importelin es mayor a la apuesta se genera)
 - 8a. `new Fig188 ( descripción : String , importelin : int )`
- (Si es menor o igual)
 - 8b. `new Fig189 ( codOpción : int , descrip : String , probab : Double , descEvento : String , codEvento : int )`



1. Se pulsa "Registrar"

2. new Fig191()

3. Se rellenan los datos y se pulsa "Registrar"

4. register (DNI: String, nombreApellidos: String, usuario: String, contraseña: String)

: boolean

5. "

6. executeSQL ("SELECT DNI FROM PERSONA WHERE Usuario :: %usuario %"): RSQZ

7. new RSQZ()

8. next(): boolean

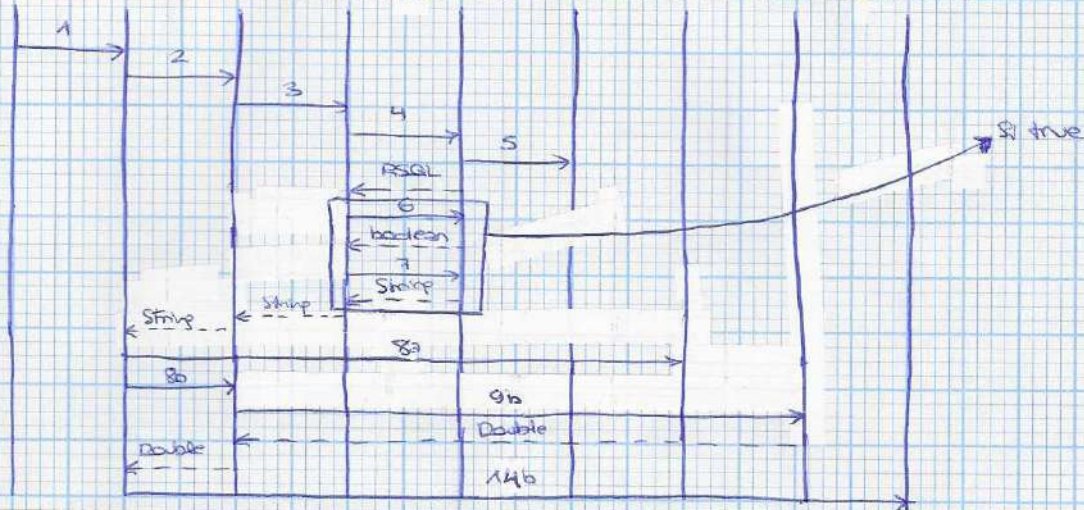
(Si false, osea no existe nadie con ese DNI)

9a. executeSQL ("INSERT INTO Persona VALUES (DNI, nombreApellidos, usuario, contraseña)"): void

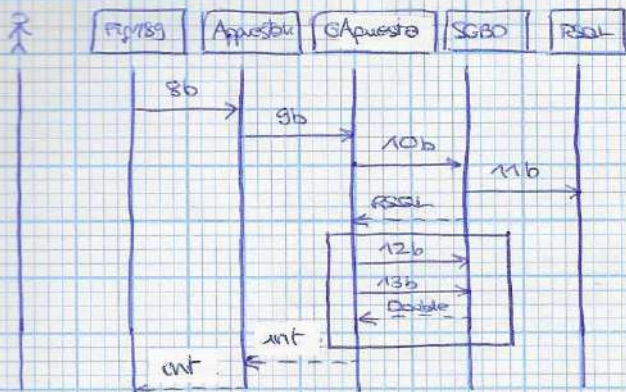
10a. new Persona (DNI: String, nombreApellidos: String, usuario: String, contraseña: String)

(Si true, osea existe alguien con ese DNI)

9b. new Fig192 (usuario: String)



1. Se introducen los datos y se pulsa "Identificar"
2. iniciar Sesión (usuario : String, contraseña : String) : boolean
3. "
4. ejecutarSQL ("SELECT DNI FROM Persona WHERE usuario: % usuario % AND
contraseña = % contraseña %") : RSQL
5. new RSQL()
6. next(): boolean
7. getString ("Dni") : String
(\$ NULL)
- 8a. new Fig190()
- (\$ NO NULL)
- 8b. getCarrito (codOpcion : int, codEvento : int, dni : String) : int
- 9b. getCarrito (codOpcion : int, codEvento : int, dni : String) : int
- 14b. new Fig193 (codOpcion : int, descripcion : String, precio : Double, descripcion :
String, apuesta : int)



8b. `getCredito (codOperación : int, codCuenta : int, dni : String ) : int`

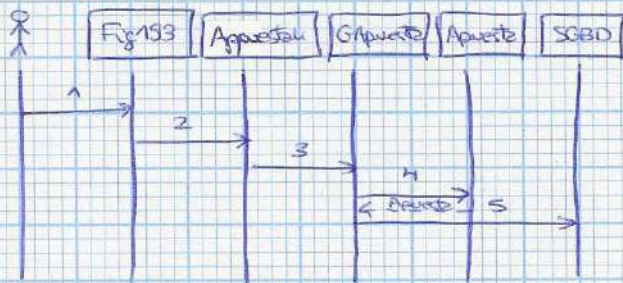
9b. "

10b. `executeSQL ("SELECT cantidadApostada FROM Apuesta NATURAL JOIN Operacion WHERE DNIPersona <> '%DNIPersona' AND codCuenta = '%codCuenta%' AND codOperación <> '%codOperación%'") : RSOL`

11b. `new RSOL()`

12b. `next(): boolean`

13b. `getDouble ("cantidadApostada") : int`



1. Pasa "Aceptar"

2. `añadirApuesta (dni : String, codOperación : int, apuesta : int, cobrar : Double) : void`

3. "

4. `new Apuesta (dni : String, codOperación : int, fechaHora : DateTime, apuesta : int, cobrar : Double)`

5. `executeSQL ("INSERT INTO Apuesta VALUES (dni'%', codOperación'%', '%row()'%', '%apuesta'%', '%cobrar'%')") : void`

Usuario	
PK	usuario contraseña nom y Apels

Vía	
PK	código Vía nombre nivel
FK1	código lugar

Lugar	
PK	código lugar nombre descripción dirección

Comentario	
PK1, FK1	USUARIO
PK2, FK2	código Vía
PK3	fecha hora texto

Completado	
PK1, FK1	USUARIO
PK2, FK2	código Vía

EscalAPP	
<<constructor>> + EscalAPP()	
+ iniciar Sesión (pusuario: String, ppassword: String): boolean	

Gestor Usuario	
- List Usuario: List<Usuario>	
<<constructor>> + Gestor Usuario()	
+ iniciar Sesión (pusuario: String, ppassword: String): boolean	

Usuario	
- usuario: String	
- contraseña: String	
- nom y Apels: String	
<<constructor>> + Usuario (puser: String, ppassword: String, pnom y Apels: String)	
+ es Usuario (pusuario: String): boolean	
+ comprobar Contraseña (ppassword: String): boolean	



Fig204

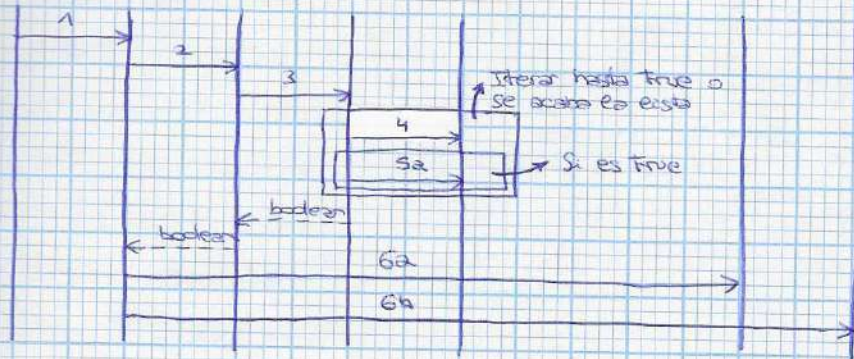
ESQAAPP

G008*

Usuario

Fig205

Fig206



1. El usuario introduce su nombre de usuario y contraseña y pulsa en "Iniciar Sesión"

2. iniciarSesion (pusuario: String, ppassword: String): boolean

3. iniciarSesion (pusuario: String, ppassword: String): boolean

4. esUsuario (pusuario: String): boolean

[Si "True"]

5a. comprobarContraseña (ppassword: String): boolean

[Si "True", se devuelve "True", si no "False"]

[Si "False"]

Se pasa al siguiente usuario

[Si "False"]

6a. new Fig205 ()

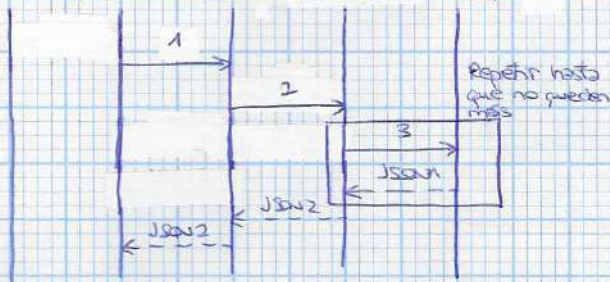
[Si "True"]

6b. new Fig206 (pusuario: String, ppassword: String)

" SELECT usuario FROM Usuario WHERE usuario = '% pusuario%' AND
 contraseña = '% ppassword%' "



Fig206 | GcdAPP | Glupar | Wpar



1. getUpares (pBúsqueda : String) : JSw2
2. getUpares (pBúsqueda : String) : JSw2
3. getData() : JSw1

JSw1 = { nombre : String, descripción : String, dirección : String }

JSw2 = { [JSw1, JSw1, ...] }

* pBúsqueda valor nullo y si es NULL no habra Retros



Fig206 | Fig207 | GcdAPP | Glupar | Wpar | Via | GcdAPP | JSw

